

CHƯƠNG 11. HUẤN LUYỆN MẠNG NƠ-RON SÂU

Trong Chương 10, bạn đã xây dựng, huấn luyện và tinh chỉnh mạng nơ-ron nhân tạo đầu tiên của mình. Nhưng đó là những mạng nông, chỉ với một vài lớp ẩn. Điều gì sẽ xảy ra nếu bạn cần giải quyết một vấn đề phức tạp, chẳng hạn như phát hiện hàng trăm loại đối tượng trong hình ảnh độ phân giải cao? Bạn có thể cần huấn luyện một ANN sâu hơn nhiều, có lẽ với 10 lớp hoặc nhiều hơn nữa, mỗi lớp chứa hàng trăm nơ-ron, được liên kết bởi hàng trăm nghìn kết nối. Huấn luyện một mạng nơ-ron sâu không hề dễ dàng. Dưới đây là một số vấn đề bạn có thể gặp phải:

- Bạn có thể phải đối mặt với vấn đề gradient ngày càng nhỏ hoặc lớn hơn, khi truyền ngược qua DNN trong quá trình huấn luyện. Cả hai vấn đề này đều làm cho các lớp thấp hơn rất khó huấn luyện.
- Bạn có thể không có đủ dữ liệu huấn luyện cho một mạng lớn như vậy, hoặc việc gán nhãn có thể quá tốn kém.
- Quá trình huấn luyện có thể cực kỳ chậm.
- Một mô hình với hàng triệu tham số sẽ có nguy cơ cao bị overfitting tập huấn luyện, đặc biệt nếu không có đủ thể hiện huấn luyện hoặc nếu chúng quá nhiều.

Trong chương này, chúng ta sẽ xem xét từng vấn đề này và trình bày các kỹ thuật để giải quyết chúng. Chúng ta sẽ bắt đầu bằng cách khám phá các vấn đề gradient biến mất và bùng nổ và một số giải pháp phổ biến nhất của chúng. Tiếp theo, chúng ta sẽ xem xét học chuyển giao và tiền huấn luyện không giám sát, có thể giúp bạn giải quyết các tác vụ phức tạp ngay cả khi bạn có ít dữ liệu được gán nhãn. Sau đó, chúng ta sẽ thảo luận về các bộ tối ưu hóa khác nhau có thể tăng tốc độ huấn luyện các mô hình lớn đáng kể. Cuối cùng, chúng ta sẽ đề cập đến một vài kỹ thuật chính quy hóa phổ biến cho các mạng nơ-ron lớn. Với những công cụ này, bạn sẽ có thể huấn luyện các mạng rất sâu. Chào mừng đến với học sâu!

Vấn đề Gradient biến mất/bùng nổ

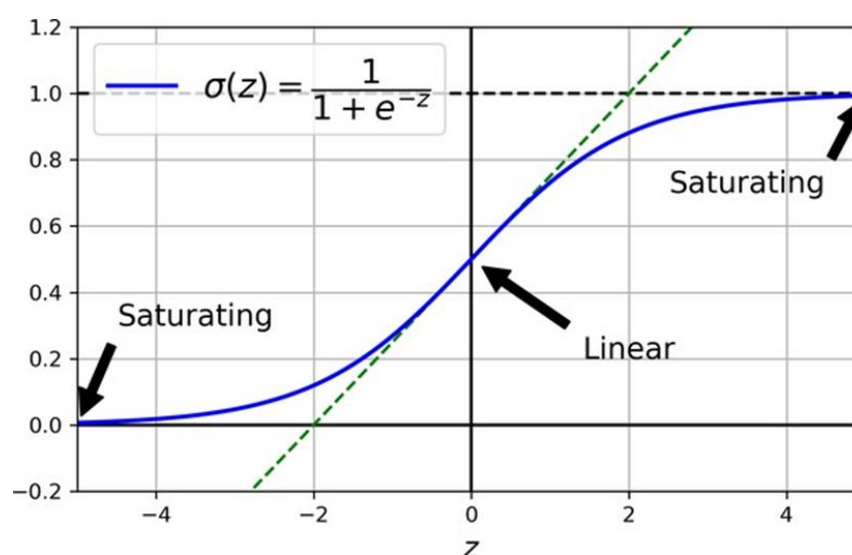
Như đã thảo luận trong Chương 10, giai đoạn thứ hai của thuật toán backpropagation hoạt động bằng cách đi từ lớp đầu ra đến lớp đầu vào, truyền gradient lỗi trên đường đi. Khi thuật toán đã tính toán gradient của hàm chi phí đối với từng tham số trong mạng, nó sử dụng các gradient này để cập nhật từng tham số bằng một bước giảm độ dốc.

Thật không may, gradient thường trở nên nhỏ hơn và nhỏ hơn khi thuật toán tiến xuống các lớp thấp hơn. Do đó, bản cập nhật giảm độ dốc làm cho trọng số kết nối của các lớp thấp hơn hầu như không thay đổi, và quá trình huấn luyện không bao giờ hội tụ đến một giải pháp tốt. Đây được gọi là **vấn đề gradient biến mất**.

Trong một số trường hợp, điều ngược lại có thể xảy ra: gradient có thể ngày càng lớn hơn cho đến khi các lớp nhận được các bản cập nhật trọng số lớn bất thường và thuật toán phân kỳ. Đây là **vấn đề gradient bùng nổ**, thường xuất hiện nhất trong các mạng nơ-ron hồi quy (xem Chương 15). Tổng quát hơn, các mạng nơ-ron sâu gặp phải các gradient không ổn định; các lớp khác nhau có thể học ở tốc độ rất khác nhau.

Hành vi không may này đã được quan sát theo kinh nghiệm từ lâu, và đó là một trong những lý do khiến mạng nơ-ron sâu phần lớn bị bỏ rơi vào đầu những năm 2000. Không rõ điều gì gây ra các gradient không ổn định như vậy khi huấn luyện DNN, nhưng một số ánh sáng đã được làm sáng tỏ trong một bài báo năm 2010 của Xavier Glorot và Yoshua Bengio. Các tác giả đã tìm thấy một vài nghi phạm, bao gồm sự kết hợp của hàm kích hoạt sigmoid (logistic) phổ biến và kỹ thuật khởi tạo trọng số phổ biến nhất vào thời điểm đó (tức là phân phối chuẩn với trung bình 0 và độ lệch chuẩn 1). Tóm lại, họ chỉ ra rằng với hàm kích hoạt này và sơ đồ khởi tạo này, phương sai của đầu ra của mỗi lớp lớn hơn nhiều so với phương sai của đầu vào của nó. Đi tiếp trong mạng, phương sai tiếp tục tăng sau mỗi lớp cho đến khi hàm kích hoạt bão hòa ở các lớp trên cùng. Sự bão hòa này thực sự trở nên tồi tệ hơn bởi thực tế là hàm sigmoid có trung bình 0.5, không phải 0 (hàm hyperbolic tangent có trung bình 0 và hoạt động tốt hơn một chút so với hàm sigmoid trong các mạng sâu).

Nhìn vào hàm kích hoạt sigmoid (xem Hình 11-1), bạn có thể thấy rằng khi đầu vào trở nên lớn (âm hoặc dương), hàm bão hòa ở 0 hoặc 1, với đạo hàm cực kỳ gần 0 (tức là đường cong phẳng ở cả hai thái cực). Do đó, khi backpropagation bắt đầu, nó hầu như không có gradient nào để truyền ngược qua mạng, và gradient ít ỏi tồn tại tiếp tục bị pha loãng khi backpropagation tiến xuống các lớp trên cùng, vì vậy thực sự không còn gì cho các lớp thấp hơn.



Hình 11-1. Sự bão hòa của hàm kích hoạt Sigmoid

Khởi tạo Glorot và He

Trong bài báo của họ, Glorot và Bengio đề xuất một cách để giảm đáng kể vấn đề gradient không ổn định. Họ chỉ ra rằng chúng ta cần tín hiệu truyền đúng cách theo cả hai hướng: theo hướng tiến khi đưa ra dự đoán và theo hướng ngược lại khi backpropagating gradient. Chúng ta không muốn tín hiệu chết đi, cũng không muốn nó bùng nổ và bão hòa. Để tín hiệu truyền đúng cách, các tác giả lập luận rằng chúng ta cần phương sai của đầu ra của mỗi lớp bằng phương sai của đầu vào của nó, và chúng ta cần các gradient có phương sai bằng nhau trước và sau khi truyền qua một lớp theo hướng ngược lại (vui lòng kiểm tra bài báo nếu bạn quan

tâm đến các chi tiết toán học). Thực tế không thể đảm bảo cả hai trừ khi lớp có số lượng đầu vào và đầu ra bằng nhau (các số này được gọi là fan_in và fan_out của lớp), nhưng Glorot và Bengio đã đề xuất một sự thỏa hiệp tốt đã được chứng minh là hoạt động rất tốt trong thực tế: trọng số kết nối của mỗi lớp phải được khởi tạo ngẫu nhiên như được mô tả trong Công thức 11-1, trong đó $fan_{avg} = (fan_{in} + fan_{out})/2$. Chiến lược khởi tạo này được gọi là **khởi tạo Xavier** hoặc **khởi tạo Glorot**, theo tên tác giả đầu tiên của bài báo.

Công thức 11-1. Khởi tạo Glorot (khi sử dụng hàm kích hoạt sigmoid)

Phân phối chuẩn với trung bình 0 và phương sai $\sigma^2 = \frac{1}{fan_{avg}}$

Hoặc phân phối đều giữa $-r$ và $+r$, với $r = \sqrt{\frac{3}{fan_{avg}}}$

Nếu bạn thay thế fan_{avg} bằng fan_{in} trong Công thức 11-1, bạn sẽ nhận được một chiến lược khởi tạo mà Yann LeCun đã đề xuất vào những năm 1990. Ông gọi đó là **khởi tạo LeCun**. Genevieve Orr và Klaus-Robert Müller thậm chí còn khuyến nghị nó trong cuốn sách *Neural Networks: Tricks of the Trade* (Springer) năm 1998 của họ. Khởi tạo LeCun tương đương với khởi tạo Glorot khi $r = \sqrt{3\sigma^2}$. Phải mất hơn một thập kỷ để các nhà nghiên cứu nhận ra mẹo này quan trọng đến mức nào. Sử dụng khởi tạo Glorot có thể tăng tốc đáng kể quá trình huấn luyện, và nó là một trong những thực hành đã dẫn đến thành công của học sâu.

Một số bài báo đã cung cấp các chiến lược tương tự cho các hàm kích hoạt khác nhau. Các chiến lược này chỉ khác nhau về thang đo phương sai và liệu chúng có sử dụng fan_{avg} hay fan_{in} hay không, như thể hiện trong Bảng 11-1 (đối với phân phối đều, chỉ cần sử dụng $r = \sqrt{3\sigma^2}$). Chiến lược khởi tạo được đề xuất cho hàm kích hoạt ReLU và các biến thể của nó được gọi là **khởi tạo He** hoặc **khởi tạo Kaiming**, theo tên tác giả đầu tiên của bài báo. Đối với SELU, hãy sử dụng phương pháp khởi tạo của Yann LeCun, tốt nhất là với phân phối chuẩn. Chúng ta sẽ đề cập đến tất cả các hàm kích hoạt này ngay sau đây.

Bảng 11-1. Các tham số khởi tạo cho từng loại hàm kích hoạt

Khởi tạo	Hàm kích hoạt	σ^2 (Chuẩn)
Glorot	None, tanh, sigmoid, softmax	$1/fan_{avg}$
He	ReLU, Leaky ReLU, ELU, GELU, Swish, Mish	$2/fan_{in}$
LeCun	SELU	$1/fan_{in}$

Theo mặc định, Keras sử dụng khởi tạo Glorot với phân phối đều. Khi bạn tạo một lớp, bạn có thể chuyển sang khởi tạo He bằng cách đặt `kernel_initializer="he_uniform"` hoặc `kernel_initializer="he_normal"` như sau:

```
import tensorflow as tf

dense = tf.keras.layers.Dense(50, activation="relu", kernel_initializer="he_normal")
```

Ngoài ra, bạn có thể nhận được bất kỳ khởi tạo nào được liệt kê trong Bảng 11-1 và hơn thế nữa bằng cách sử dụng trình khởi tạo `VarianceScaling`. Ví dụ, nếu bạn muốn khởi tạo He với phân phối đều và dựa trên fan_{avg} (thay vì fan_{in}), bạn có thể sử dụng đoạn mã sau:

```
he_avg_init = tf.keras.initializers.VarianceScaling(scale=2., mode="fan_avg",
distribution="uniform")
dense = tf.keras.layers.Dense(50, activation="sigmoid", kernel_initializer=he_avg_init)
```

Các hàm kích hoạt tốt hơn

Một trong những nhận thức trong bài báo năm 2010 của Glorot và Bengio là các vấn đề về gradient không ổn định một phần là do lựa chọn hàm kích hoạt kém. Cho đến lúc đó, hầu hết mọi người đều cho rằng nếu Mẹ Thiên nhiên đã chọn sử dụng các hàm kích hoạt gần như sigmoid trong các nơ-ron sinh học, thì chúng phải là một lựa chọn tuyệt vời. Nhưng hóa ra các hàm kích hoạt khác hoạt động tốt hơn nhiều trong mạng nơ-ron sâu — đặc biệt là hàm kích hoạt ReLU, chủ yếu vì nó không bão hòa đối với các giá trị dương, và cũng vì nó rất nhanh để tính toán.

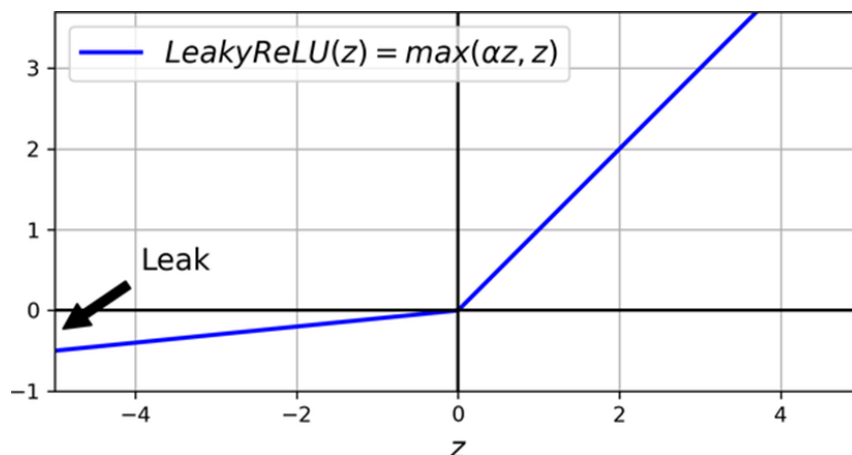
Thật không may, hàm kích hoạt ReLU không hoàn hảo. Nó gặp phải một vấn đề được gọi là **ReLU chết (dying ReLUs)**: trong quá trình huấn luyện, một số nơ-ron thực sự “chết”, có nghĩa là chúng ngừng xuất ra bất cứ thứ gì khác ngoài 0. Trong một số trường hợp, bạn có thể thấy rằng một nửa số nơ-ron của mạng đã chết, đặc biệt nếu bạn sử dụng tốc độ học lớn. Một nơ-ron chết khi trọng số của nó được điều chỉnh theo cách mà đầu vào của hàm ReLU (tức là tổng có trọng số của đầu vào của nơ-ron cộng với số hạng độ lệch của nó) là âm đối với tất cả các thể hiện trong tập huấn luyện. Khi điều này xảy ra, nó chỉ tiếp tục xuất ra các số 0, và gradient độ dốc không còn ảnh hưởng đến nó nữa vì gradient của hàm ReLU bằng 0 khi đầu vào của nó là âm.

Để giải quyết vấn đề này, bạn có thể muốn sử dụng một biến thể của hàm ReLU, chẳng hạn như leaky ReLU.

Leaky ReLU

Hàm kích hoạt leaky ReLU được định nghĩa là $\text{leakyReLU}_{\alpha}(z) = \max(\alpha z, z)$ (xem Hình 11-2). Siêu tham số α định nghĩa mức độ hàm “rò rỉ”: đó là độ dốc của hàm đối với $z < 0$. Có một độ dốc đối với $z < 0$ đảm bảo rằng leaky ReLU không bao giờ chết; chúng có thể rơi vào tình trạng hôn mê dài, nhưng chúng có cơ hội cuối cùng thức dậy. Một bài báo năm 2015 của Bing Xu và cộng sự đã so sánh một số biến thể của hàm kích hoạt ReLU, và một trong những kết luận của nó là các biến thể rò rỉ luôn hoạt động tốt hơn hàm kích hoạt ReLU nghiêm ngặt. Trên thực tế, đặt $\alpha = 0.2$ (một rò rỉ lớn) dường như mang lại hiệu suất tốt hơn so với $\alpha = 0.01$ (một rò rỉ nhỏ). Bài báo cũng đánh giá randomized leaky ReLU (RReLU), trong đó α được chọn ngẫu nhiên trong một phạm vi nhất định trong quá trình huấn luyện và được cố định thành một giá trị trung bình trong quá trình kiểm tra. RReLU cũng hoạt động khá tốt và dường như hoạt động như một bộ chính quy hóa, giảm nguy cơ overfitting tập huấn luyện. Cuối cùng, bài báo đã đánh giá parametric leaky ReLU (PReLU), trong đó α được phép học trong quá trình huấn luyện: thay vì là một siêu tham số, nó trở

thành một tham số có thể được sửa đổi bằng backpropagation như bất kỳ tham số nào khác. PReLU được báo cáo là hoạt động tốt hơn đáng kể so với ReLU trên các tập dữ liệu hình ảnh lớn, nhưng trên các tập dữ liệu nhỏ hơn, nó có nguy cơ overfitting tập huấn luyện.



Hình 11-2. Leaky ReLU: giống như ReLU, nhưng có độ dốc nhỏ cho các giá trị âm

Keras bao gồm các lớp LeakyReLU và PReLU trong gói `tf.keras.layers`. Giống như các biến thể ReLU khác, bạn nên sử dụng khởi tạo He với chúng. Ví dụ:

```
leaky_relu = tf.keras.layers.LeakyReLU(alpha=0.2) # defaults to alpha=0.3
dense = tf.keras.layers.Dense(50, activation=leaky_relu, kernel_initializer="he_normal")
```

Nếu bạn thích, bạn cũng có thể sử dụng LeakyReLU làm một lớp riêng biệt trong mô hình của mình; điều này không tạo ra sự khác biệt nào cho quá trình huấn luyện và dự đoán:

```
model = tf.keras.models.Sequential([
    # ... more layers
    tf.keras.layers.Dense(50, kernel_initializer="he_normal"), # no activation
    tf.keras.layers.LeakyReLU(alpha=0.2), # activation as a separate layer
    # ... more layers
])
```

Đối với PReLU, hãy thay thế LeakyReLU bằng PReLU. Hiện tại không có triển khai chính thức của RReLU trong Keras, nhưng bạn có thể khá dễ dàng triển khai của riêng mình (để tìm hiểu cách làm điều đó, xem các bài tập ở cuối Chương 12).

ReLU, leaky ReLU và PReLU đều gặp phải vấn đề là chúng không phải là các hàm trơn: đạo hàm của chúng thay đổi đột ngột (tại $z = 0$). Như chúng ta đã thấy trong Chương 4 khi thảo luận về lasso, loại gián đoạn này có thể khiến giảm độ dốc dao động quanh cực tiểu và làm chậm quá trình hội tụ. Vì vậy, bây giờ chúng ta sẽ xem xét một số biến thể trơn của hàm kích hoạt ReLU, bắt đầu với ELU và SELU.

ELU và SELU

Vào năm 2015, một bài báo của Djork-Arné Clevert và cộng sự đã đề xuất một hàm kích hoạt mới, được gọi là **đơn vị tuyến tính mũ (ELU)**, đã hoạt động tốt hơn tất cả các biến thể ReLU trong các thí nghiệm của các tác giả: thời gian huấn luyện được giảm, và mạng nơ-ron hoạt động tốt hơn trên tập kiểm tra. Công thức 11-2 cho thấy định nghĩa của hàm kích hoạt này.

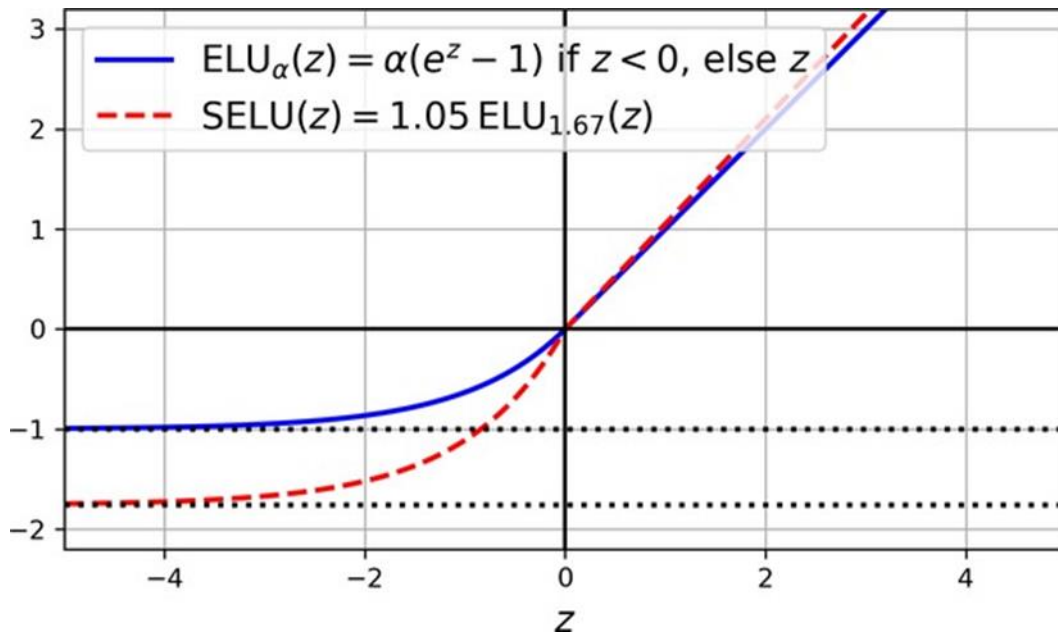
Công thức 11-2. Hàm kích hoạt ELU

$$\text{ELU}_\alpha(z) = \begin{cases} \alpha(\exp(z) - 1) & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$$

Hàm kích hoạt ELU trông rất giống hàm ReLU (xem Hình 11-3), với một vài khác biệt lớn:

- Nó nhận các giá trị âm khi $z < 0$, điều này cho phép đơn vị có đầu ra trung bình gần 0 hơn và giúp giảm nhẹ vấn đề gradient biến mất. Siêu tham số *alpha* định nghĩa ngược lại giá trị mà hàm ELU tiến tới khi z là một số âm lớn. Nó thường được đặt là 1, nhưng bạn có thể điều chỉnh nó như bất kỳ siêu tham số nào khác.
- Nó có một gradient khác 0 cho $z < 0$, điều này tránh được vấn đề nơ-ron chết.
- Nếu *alpha* bằng 1 thì hàm trơn ở khắp mọi nơi, bao gồm cả quanh $z = 0$, điều này giúp tăng tốc giảm độ dốc vì nó không dao động nhiều sang trái và phải của $z = 0$.

Sử dụng ELU với Keras dễ dàng như đặt `activation="elu"`, và giống như các biến thể ReLU khác, bạn nên sử dụng khởi tạo He. Nhược điểm chính của hàm kích hoạt ELU là nó chậm hơn khi tính toán so với hàm ReLU và các biến thể của nó (do sử dụng hàm mũ). Tốc độ hội tụ nhanh hơn trong quá trình huấn luyện có thể bù đắp cho việc tính toán chậm đó, nhưng dù sao, ở thời điểm kiểm tra, một mạng ELU sẽ chậm hơn một chút so với một mạng ReLU.



Hình 11-3. Các hàm kích hoạt ELU và SELU

Không lâu sau đó, một bài báo năm 2017 của Günter Klambauer và cộng sự đã giới thiệu hàm kích hoạt **SELU (scaled ELU)**: đúng như tên gọi của nó, đây là một biến thể được điều chỉnh theo tỷ lệ của hàm kích hoạt ELU (khoảng 1.05 lần ELU, sử dụng $\alpha \approx 1.67$). Các tác giả đã chỉ ra rằng nếu bạn xây dựng một mạng nơ-ron chỉ bao gồm một chồng các lớp dày đặc (tức là một MLP), và nếu tất cả các lớp ẩn đều sử dụng hàm kích hoạt SELU, thì mạng sẽ tự chuẩn hóa: đầu ra của mỗi lớp sẽ có xu hướng giữ trung bình 0 và độ lệch chuẩn 1 trong quá trình huấn luyện, điều này giải quyết vấn đề gradient biến mất/bùng nổ. Kết quả là, hàm kích hoạt SELU có thể hoạt động tốt hơn các hàm kích hoạt khác cho MLP, đặc biệt là các mạng sâu. Để sử dụng nó với Keras, chỉ cần đặt `activation="selu"`. Tuy nhiên, có một vài điều kiện để tự chuẩn hóa xảy ra (xem bài báo để biết lý do toán học):

- Các đặc trưng đầu vào phải được chuẩn hóa: trung bình 0 và độ lệch chuẩn 1.
- Trọng số của mỗi lớp ẩn phải được khởi tạo bằng khởi tạo chuẩn LeCun. Trong Keras, điều này có nghĩa là đặt `kernel_initializer="lecun_normal"`.
- Thuộc tính tự chuẩn hóa chỉ được đảm bảo với các MLP đơn giản. Nếu bạn cố gắng sử dụng SELU trong các kiến trúc khác, như mạng hồi quy (xem Chương 15) hoặc mạng có kết nối bỏ qua (tức là các kết nối bỏ qua các lớp, chẳng hạn như trong mạng Wide & Deep), nó có lẽ sẽ không hoạt động tốt hơn ELU.
- Bạn không thể sử dụng các kỹ thuật chính quy hóa như chính quy hóa l_1 hoặc l_2 , max-norm, batch-norm hoặc dropout thông thường (những điều này sẽ được thảo luận sau trong chương này).

Đây là những ràng buộc đáng kể, vì vậy mặc dù có những hứa hẹn, SELU đã không được chấp nhận rộng rãi. Hơn nữa, ba hàm kích hoạt khác dường như hoạt động tốt hơn nó một cách khá nhất quán trên hầu hết các tác vụ: GELU, Swish và Mish.

GELU, Swish và Mish

GELU được giới thiệu trong một bài báo năm 2016 của Dan Hendrycks và Kevin Gimpel. Một lần nữa, bạn có thể coi nó là một biến thể trơn của hàm kích hoạt ReLU. Định nghĩa của nó được đưa ra trong Công thức 11-3, trong đó Φ là hàm phân phối tích lũy Gaussian chuẩn (CDF): $\Phi(z)$ tương ứng với xác suất mà một giá trị được lấy mẫu ngẫu nhiên từ phân phối chuẩn có trung bình 0 và phương sai 1 thấp hơn z .

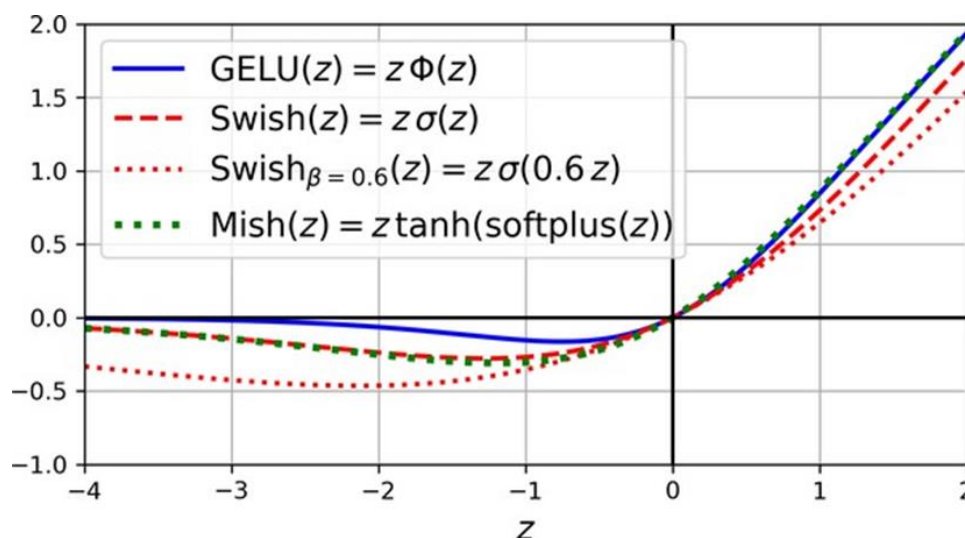
Công thức 11-3. Hàm kích hoạt GELU

$$GELU(z) = z\Phi(z)$$

Như bạn có thể thấy trong Hình 11-4, GELU giống như ReLU: nó tiến về 0 khi đầu vào z rất âm, và nó tiến về z khi z rất dương. Tuy nhiên, trong khi tất cả các hàm kích hoạt chúng ta đã thảo luận cho đến nay đều vừa lồi vừa đơn điệu, hàm kích hoạt GELU thì không: từ trái sang phải, nó bắt đầu đi thẳng, sau đó nó uốn lượn xuống, đạt đến điểm thấp nhất khoảng -0.17 (gần $z \approx 0.75$), và cuối cùng bật lên và kết thúc đi thẳng lên phía trên bên phải. Hình dạng khá phức tạp này và thực tế là nó có độ cong ở mọi điểm có thể giải thích tại sao nó hoạt động rất tốt, đặc biệt đối với các tác vụ phức tạp: giảm độ dốc có thể thấy dễ dàng hơn để

phù hợp với các mẫu phức tạp. Trong thực tế, nó thường hoạt động tốt hơn mọi hàm kích hoạt khác đã được thảo luận cho đến nay.

Tuy nhiên, nó tốn nhiều tính toán hơn một chút, và sự tăng cường hiệu suất mà nó cung cấp không phải lúc nào cũng đủ để biện minh cho chi phí bổ sung. Điều đó nói lên rằng, có thể chứng minh rằng nó xấp xỉ bằng $z\sigma(1.702z)$, trong đó σ là hàm sigmoid: sử dụng xấp xỉ này cũng hoạt động rất tốt, và nó có ưu điểm là tính toán nhanh hơn nhiều.



Hình 11-4. Các hàm kích hoạt GELU, Swish, Swish tham số hóa và Mish

Bài báo GELU cũng giới thiệu hàm kích hoạt đơn vị tuyến tính sigmoid (SiLU), bằng $z\sigma(z)$, nhưng nó đã bị GELU vượt trội trong các thử nghiệm của các tác giả.

Điều thú vị là, một bài báo năm 2017 của Prajit Ramachandran và cộng sự đã khám phá lại hàm SiLU bằng cách tự động tìm kiếm các hàm kích hoạt tốt. Các tác giả đã đặt tên nó là Swish, và cái tên này đã được chấp nhận. Trong bài báo của họ, Swish đã vượt trội hơn mọi hàm khác, bao gồm cả GELU. Ramachandran và cộng sự sau đó đã tổng quát hóa Swish bằng cách thêm một siêu tham số β để điều chỉnh đầu vào của hàm sigmoid. Hàm Swish tổng quát là $Swish_{\beta}(z) = z\sigma(\beta z)$, vì vậy GELU xấp xỉ bằng hàm Swish tổng quát sử dụng $\beta = 1.702$. Bạn có thể điều chỉnh β như bất kỳ siêu tham số nào khác. Ngoài ra, cũng có thể làm cho β có thể huấn luyện và để giảm độ dốc tối ưu hóa nó: giống như PReLU, điều này có thể làm cho mô hình của bạn mạnh mẽ hơn, nhưng nó cũng có nguy cơ overfitting dữ liệu.

Một hàm kích hoạt khá tương tự khác là Mish, được giới thiệu trong một bài báo năm 2019 của Diganta Misra. Nó được định nghĩa là $mish(z) = z \tanh(\text{softplus}(z))$, trong đó $\text{softplus}(z) = \log(1 + \exp(z))$. Giống như GELU và Swish, nó là một biến thể trơn, không lỗi và không đơn điệu của ReLU, và một lần nữa tác giả đã thực hiện nhiều thí nghiệm và thấy rằng Mish nói chung hoạt động tốt hơn các hàm kích hoạt khác - thậm chí cả Swish và GELU, với một biên độ nhỏ. Hình 11-4 hiển thị GELU, Swish (cả với $\beta = 1$ mặc định và với $\beta = 0.6$), và cuối cùng là Mish. Như bạn có thể thấy, Mish chồng lên gần như hoàn hảo với Swish khi z âm, và gần như hoàn hảo với GELU khi z dương.

Keras hỗ trợ GELU và Swish ngay lập tức; chỉ cần sử dụng `activation="gelu"` hoặc `activation="swish"`. Tuy nhiên, nó chưa hỗ trợ Mish hoặc hàm kích hoạt Swish tổng quát (nhưng xem Chương 12 để biết cách triển khai các hàm kích hoạt và lớp của riêng bạn).

Đó là tất cả về các hàm kích hoạt! Bây giờ, hãy xem xét một cách hoàn toàn khác để giải quyết vấn đề gradient không ổn định: chuẩn hóa theo batch (batch normalization).

Chuẩn hóa theo Batch

Mặc dù sử dụng khởi tạo He cùng với ReLU (hoặc bất kỳ biến thể nào của nó) có thể giảm đáng kể nguy cơ của các vấn đề gradient biến mất/bùng nổ khi bắt đầu huấn luyện, nhưng nó không đảm bảo rằng chúng sẽ không quay trở lại trong quá trình huấn luyện.

Trong một bài báo năm 2015, Sergey Ioffe và Christian Szegedy đã đề xuất một kỹ thuật được gọi là **chuẩn hóa theo batch (BN)** giải quyết các vấn đề này. Kỹ thuật này bao gồm việc thêm một thao tác vào mô hình ngay trước hoặc sau hàm kích hoạt của mỗi lớp ẩn. Thao tác này chỉ đơn giản là đưa về tâm 0 và chuẩn hóa từng đầu vào, sau đó điều chỉnh tỷ lệ và dịch chuyển kết quả bằng cách sử dụng hai vector tham số mới cho mỗi lớp: một cho điều chỉnh tỷ lệ, một cho dịch chuyển. Nói cách khác, thao tác này cho phép mô hình học được tỷ lệ và trung bình tối ưu của từng đầu vào của lớp. Trong nhiều trường hợp, nếu bạn thêm một lớp BN làm lớp đầu tiên của mạng nơ-ron của bạn, bạn không cần phải chuẩn hóa tập huấn luyện của mình. Tức là, không cần StandardScaler hoặc Normalization; lớp BN sẽ làm điều đó cho bạn (tất nhiên, gần đúng, vì nó chỉ xem xét từng batch một, và nó cũng có thể điều chỉnh tỷ lệ và dịch chuyển từng đặc trưng đầu vào).

Để đưa về tâm 0 và chuẩn hóa các đầu vào, thuật toán cần ước tính trung bình và độ lệch chuẩn của từng đầu vào. Nó làm như vậy bằng cách đánh giá trung bình và độ lệch chuẩn của đầu vào trên mini-batch hiện tại (do đó có tên “chuẩn hóa theo batch”). Toàn bộ thao tác được tóm tắt từng bước trong Công thức 11-4.

Công thức 11-4. Thuật toán chuẩn hóa theo batch

$$\begin{aligned} 1. \quad \mu_B &= \frac{1}{m_B} \sum_{i=1}^{m_B} \mathbf{x}^{(i)} \\ 2. \quad \sigma_B^2 &= \frac{1}{m_B} \sum_{i=1}^{m_B} \left(\mathbf{x}^{(i)} - \mu_B \right)^2 \\ 3. \quad \hat{\mathbf{x}}^{(i)} &= \frac{\mathbf{x}^{(i)} - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} \\ 4. \quad \mathbf{z}^{(i)} &= \gamma \otimes \hat{\mathbf{x}}^{(i)} + \beta \end{aligned}$$

Trong thuật toán này:

- μ_B là **vector trung bình đầu vào**, được tính trên toàn bộ mini-batch B (nó chứa một giá trị trung bình cho mỗi đầu vào).
- m_B là **số lượng trường hợp** trong mini-batch.

- σ_B là **vector độ lệch chuẩn đầu vào**, cũng được tính trên toàn bộ mini-batch (nó chứa một độ lệch chuẩn cho mỗi đầu vào).
- $x^{(i)}$ là **vector đầu vào đã được chuẩn hóa về 0** và được chuẩn hóa cho trường hợp i .
- ϵ là một số rất nhỏ để **tránh phép chia cho 0** và đảm bảo gradient không trở nên quá lớn (thường là 10^{-5}). Số này được gọi là **số hạng làm mịn** (smoothing term).
- γ là **vector tham số tỷ lệ đầu ra** cho lớp (nó chứa một tham số tỷ lệ cho mỗi đầu vào).

Vì vậy, trong quá trình huấn luyện, BN chuẩn hóa các đầu vào của nó, sau đó điều chỉnh tỷ lệ và bù đắp chúng. Tốt! Còn ở thời điểm kiểm tra thì sao? Chà, không đơn giản như vậy. Thật vậy, chúng ta có thể cần đưa ra dự đoán cho các thể hiện riêng lẻ thay vì cho các batch của thể hiện: trong trường hợp này, chúng ta sẽ không có cách nào để tính toán trung bình và độ lệch chuẩn của mỗi đầu vào. Hơn nữa, ngay cả khi chúng ta có một batch các thể hiện, nó có thể quá nhỏ, hoặc các thể hiện có thể không độc lập và phân phối giống nhau, vì vậy việc tính toán thống kê trên các thể hiện batch sẽ không đáng tin cậy. Một giải pháp có thể là đợi đến cuối quá trình huấn luyện, sau đó chạy toàn bộ tập huấn luyện qua mạng nơ-ron và tính toán trung bình và độ lệch chuẩn của mỗi đầu vào của lớp BN. Các trung bình và độ lệch chuẩn đầu vào “cuối cùng” này sau đó có thể được sử dụng thay cho trung bình và độ lệch chuẩn đầu vào batch khi đưa ra dự đoán. Tuy nhiên, hầu hết các triển khai chuẩn hóa theo batch ước tính các thống kê cuối cùng này trong quá trình huấn luyện bằng cách sử dụng trung bình động của trung bình đầu vào và độ lệch chuẩn của lớp. Đây là điều Keras tự động làm khi bạn sử dụng lớp `BatchNormalization`. Tóm lại, bốn vector tham số được học trong mỗi lớp chuẩn hóa theo batch: *gamma* (vector tỷ lệ đầu ra) và *beta* (vector bù đắp đầu ra) được học thông qua backpropagation thông thường, và *mu* (vector trung bình đầu vào cuối cùng) và *sigma* (vector độ lệch chuẩn đầu vào cuối cùng) được ước tính bằng cách sử dụng trung bình động mũ. Lưu ý rằng *mu* và *sigma* được ước tính trong quá trình huấn luyện, nhưng chúng chỉ được sử dụng sau khi huấn luyện (để thay thế trung bình đầu vào batch và độ lệch chuẩn trong Công thức 11-4).

Ioffe và Szegedy đã chứng minh rằng chuẩn hóa theo batch cải thiện đáng kể tất cả các mạng nơ-ron sâu mà họ đã thử nghiệm, dẫn đến một cải thiện lớn trong tác vụ phân loại ImageNet (ImageNet là một cơ sở dữ liệu lớn các hình ảnh được phân loại thành nhiều lớp, thường được sử dụng để đánh giá các hệ thống thị giác máy tính). Vấn đề gradient biến mất đã được giảm mạnh, đến mức họ có thể sử dụng các hàm kích hoạt bão hòa như tanh và thậm chí cả hàm kích hoạt sigmoid. Các mạng cũng ít nhạy cảm hơn nhiều với khởi tạo trọng số. Các tác giả đã có thể sử dụng tốc độ học lớn hơn nhiều, tăng tốc đáng kể quá trình học. Cụ thể, họ lưu ý rằng: Áp dụng cho một mô hình phân loại hình ảnh hiện đại, chuẩn hóa theo batch đạt được cùng độ chính xác với số bước huấn luyện ít hơn 14 lần, và vượt trội mô hình gốc với một biên độ đáng kể. [...] Sử dụng một tập hợp các mạng được chuẩn hóa theo batch, chúng tôi cải thiện kết quả tốt nhất đã được công bố về phân loại ImageNet: đạt 4.9% lỗi xác thực top-5 (và 4.8% lỗi kiểm tra), vượt quá độ chính xác của người đánh giá.

Cuối cùng, như một món quà tiếp tục mang lại, chuẩn hóa theo batch hoạt động như một bộ chính quy hóa, giảm nhu cầu về các kỹ thuật chính quy hóa khác (chẳng hạn như dropout, được mô tả sau trong chương này).

May mắn thay, thường có thể **hợp nhất lớp BN** với lớp trước đó sau khi huấn luyện, từ đó tránh được chi phí khi chạy. Việc này được thực hiện bằng cách cập nhật các trọng số và độ lệch của lớp trước đó sao cho nó trực tiếp tạo ra đầu ra với tỷ lệ và độ lệch phù hợp. Ví dụ, nếu lớp trước đó tính $XW + b$, thì lớp BN sẽ tính $\gamma \otimes (XW + b - \mu)/\sigma + \beta$ (bỏ qua số hạng làm mịn ϵ ở mẫu số). Nếu chúng ta định nghĩa $W' = \gamma \otimes W/\sigma$ và $b' = \gamma \otimes (b - \mu)/\sigma + \beta$, thì phương trình sẽ được đơn giản hóa thành $XW' + b'$. Vì vậy, nếu chúng ta thay thế các trọng số và độ lệch của lớp trước đó (W và b) bằng các trọng số và độ lệch đã cập nhật (W' và b'), chúng ta có thể loại bỏ lớp BN. (Bộ chuyển đổi của TFLite thực hiện điều này một cách tự động).

Triển khai chuẩn hóa theo batch với Keras

Với hầu hết mọi thứ với Keras, việc triển khai chuẩn hóa theo batch rất đơn giản và trực quan. Chỉ cần thêm một lớp BatchNormalization trước hoặc sau hàm kích hoạt của mỗi lớp ẩn. Bạn cũng có thể thêm một lớp BN làm lớp đầu tiên trong mô hình của mình, nhưng một lớp Normalization đơn giản thường hoạt động tốt như vậy ở vị trí này (nhược điểm duy nhất của nó là bạn phải gọi phương thức `adapt()` của nó trước). Ví dụ, mô hình này áp dụng BN sau mỗi lớp ẩn và làm lớp đầu tiên trong mô hình (sau khi làm phẳng hình ảnh đầu vào):

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(300, activation="relu", kernel_initializer="he_normal"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(100, activation="relu", kernel_initializer="he_normal"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

Vậy là xong! Trong ví dụ nhỏ này chỉ với hai lớp ẩn, chuẩn hóa theo batch có thể không có tác động lớn, nhưng đối với các mạng sâu hơn, nó có thể tạo ra sự khác biệt lớn.

Hãy hiển thị tóm tắt mô hình:

```
>>> model.summary()
Model: "sequential"
```

Layer (type)	Output Shape	Param#
=====		
flatten (Flatten)	(None, 784)	0
batch_normalization (Batch Normalization)	(None, 784)	3136
dense (Dense)	(None, 300)	235500
batch_normalization_1 (Batch Normalization)	(None, 300)	1200

dense_1 (Dense)	(None, 100)	30100
batch_normalization_2 (Batch Normalization)	(None, 100)	400
dense_2 (Dense)	(None, 10)	1010

=====

Total params: 271,346
Trainable params: 268,978
Non-trainable params: 2,368

Như bạn có thể thấy, mỗi lớp BN thêm bốn tham số cho mỗi đầu vào: *gamma*, *beta*, *mu* và *sigma* (ví dụ: lớp BN đầu tiên thêm 3.136 tham số, tức là 4×784). Hai tham số cuối cùng, *mu* và *sigma*, là các trung bình di chuyển; chúng không bị ảnh hưởng bởi backpropagation, vì vậy Keras gọi chúng là “không thể huấn luyện” (nếu bạn đếm tổng số tham số BN, $3.136 + 1.200 + 400$, và chia cho 2, bạn sẽ nhận được 2.368, là tổng số tham số không thể huấn luyện trong mô hình này).

Hãy xem các tham số của lớp BN đầu tiên. Hai trong số đó có thể huấn luyện (bằng backpropagation), và hai không:

```
>>> [(var.name, var.trainable) for var in model.layers[1].variables]
[('batch_normalization/gamma:0', True),
 ('batch_normalization/beta:0', True),
 ('batch_normalization/moving_mean:0', False),
 ('batch_normalization/moving_variance:0', False)]
```

Các tác giả của bài báo BN đã ủng hộ việc thêm các lớp BN trước các hàm kích hoạt, thay vì sau (như chúng ta vừa làm). Có một số tranh cãi về điều này, vì điều nào được ưu tiên dường như phụ thuộc vào tác vụ — bạn cũng có thể thử nghiệm điều này để xem tùy chọn nào hoạt động tốt nhất trên tập dữ liệu của mình. Để thêm các lớp BN trước hàm kích hoạt, bạn phải loại bỏ các hàm kích hoạt khỏi các lớp ẩn và thêm chúng làm các lớp riêng biệt sau các lớp BN. Hơn nữa, vì một lớp chuẩn hóa theo batch bao gồm một tham số offset cho mỗi đầu vào, bạn có thể loại bỏ số hạng bias khỏi lớp trước đó bằng cách truyền `use_bias=False` khi tạo nó. Cuối cùng, bạn thường có thể bỏ qua lớp BN đầu tiên để tránh kẹp lớp ẩn đầu tiên giữa hai lớp BN. Mã đã cập nhật trông như thế này:

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dense(300, kernel_initializer="he_normal", use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation("relu"),
    tf.keras.layers.Dense(100, kernel_initializer="he_normal", use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation("relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

Lớp BatchNormalization có khá nhiều siêu tham số bạn có thể điều chỉnh. Các giá trị mặc định thường sẽ ổn, nhưng đôi khi bạn có thể cần điều chỉnh momentum. Siêu tham số này được lớp BatchNormalization sử dụng khi nó cập nhật các trung bình di chuyển mũ; với một giá trị mới v (tức là một vector mới của trung bình đầu vào hoặc độ lệch chuẩn được tính trên batch hiện tại), lớp cập nhật trung bình chạy *hatv* bằng phương trình sau:

$$\hat{v} \leftarrow \hat{v} \times \text{momentum} + v \times (1 - \text{momentum})$$

Giá trị momentum tốt thường gần 1; ví dụ, 0.9, 0.99, hoặc 0.999. Bạn muốn nhiều số 9 hơn cho các tập dữ liệu lớn hơn và cho các mini-batch nhỏ hơn.

Một siêu tham số quan trọng khác là *axis*: nó xác định trục nào nên được chuẩn hóa. Nó mặc định là -1, có nghĩa là theo mặc định nó sẽ chuẩn hóa trục cuối cùng (sử dụng trung bình và độ lệch chuẩn được tính trên các trục khác). Khi batch đầu vào là 2D (tức là hình dạng batch là [kích thước batch, đặc trưng]), điều này có nghĩa là mỗi đặc trưng đầu vào sẽ được chuẩn hóa dựa trên trung bình và độ lệch chuẩn được tính trên tất cả các thể hiện trong batch. Ví dụ, lớp BN đầu tiên trong ví dụ mã trước đó sẽ độc lập chuẩn hóa (và điều chỉnh tỷ lệ và dịch chuyển) mỗi trong số 784 đặc trưng đầu vào. Nếu chúng ta di chuyển lớp BN đầu tiên trước lớp Flatten, thì các batch đầu vào sẽ là 3D, với hình dạng [kích thước batch, chiều cao, chiều rộng]; do đó, lớp BN sẽ tính toán 28 trung bình và 28 độ lệch chuẩn (1 cho mỗi cột pixel, được tính trên tất cả các thể hiện trong batch và trên tất cả các hàng trong cột), và nó sẽ chuẩn hóa tất cả các pixel trong một cột nhất định bằng cách sử dụng cùng trung bình và độ lệch chuẩn. Cũng sẽ chỉ có 28 tham số tỷ lệ và 28 tham số dịch chuyển. Nếu thay vào đó bạn vẫn muốn xử lý từng trong số 784 pixel một cách độc lập, thì bạn nên đặt *axis*=[1, 2].

Chuẩn hóa theo batch đã trở thành một trong những lớp được sử dụng nhiều nhất trong các mạng nơ-ron sâu, đặc biệt là các mạng nơ-ron tích chập sâu được thảo luận trong (Chương 14), đến mức nó thường bị bỏ qua trong các sơ đồ kiến trúc: người ta giả định rằng BN được thêm vào sau mỗi lớp. Bây giờ chúng ta hãy xem một kỹ thuật cuối cùng để ổn định gradient trong quá trình huấn luyện: cắt gradient.

Cắt Gradient (Gradient Clipping)

Một kỹ thuật khác để giảm thiểu vấn đề gradient bùng nổ là cắt gradient trong quá trình backpropagation để chúng không bao giờ vượt quá một ngưỡng nhất định. Kỹ thuật này thường được sử dụng trong các mạng nơ-ron hồi quy, nơi việc sử dụng chuẩn hóa theo batch rất phức tạp (như bạn sẽ thấy trong Chương 15).

Trong Keras, việc triển khai cắt gradient chỉ đơn giản là đặt đối số *clipvalue* hoặc *clipnorm* khi tạo một bộ tối ưu hóa, như thế này:

```
optimizer = tf.keras.optimizers.SGD(clipvalue=1.0)
model.compile(..., optimizer=optimizer)
```

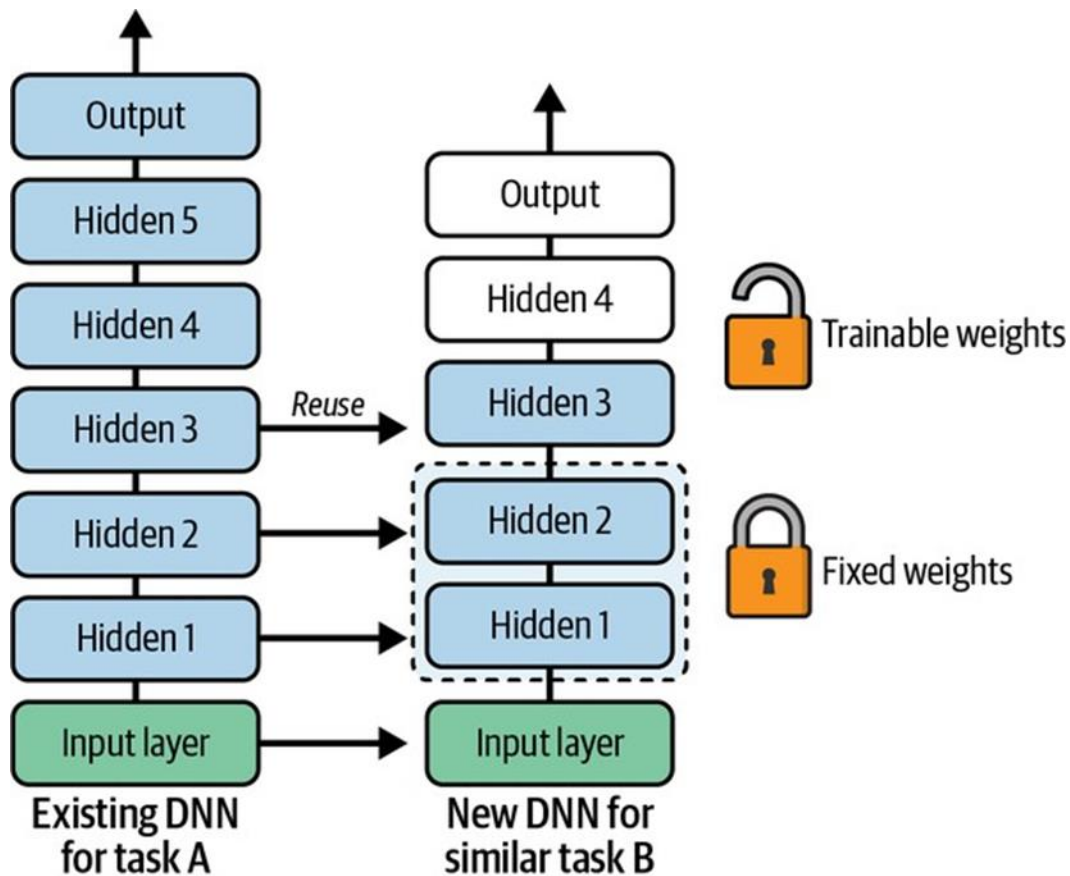
Bộ tối ưu hóa này sẽ cắt mọi thành phần của vector gradient xuống một giá trị giữa -1.0 và 1.0. Điều này có nghĩa là tất cả các đạo hàm riêng của hàm mất mát (đối với từng tham số có thể huấn luyện) sẽ được cắt giữa -1.0 và 1.0. Ngưỡng là một siêu tham số bạn có thể điều

chỉnh. Lưu ý rằng nó có thể thay đổi hướng của vector gradient. Ví dụ, nếu vector gradient gốc là $[0.9, 100.0]$, nó chủ yếu chỉ theo hướng của trục thứ hai; nhưng một khi bạn cắt nó theo giá trị, bạn sẽ nhận được $[0.9, 1.0]$, cái này chỉ gần như theo đường chéo giữa hai trục. Trong thực tế, cách tiếp cận này hoạt động tốt. Nếu bạn muốn đảm bảo rằng cắt gradient không làm thay đổi hướng của vector gradient, bạn nên cắt theo chuẩn bằng cách đặt `clipnorm` thay vì `clipvalue`. Điều này sẽ cắt toàn bộ gradient nếu chuẩn L2 của nó lớn hơn ngưỡng bạn đã chọn. Ví dụ, nếu bạn đặt `clipnorm=1.0`, thì vector $[0.9, 100.0]$ sẽ được cắt thành $[0.00899964, 0.9999595]$, giữ nguyên hướng nhưng gần như loại bỏ thành phần đầu tiên. Nếu bạn quan sát thấy các gradient bùng nổ trong quá trình huấn luyện (bạn có thể theo dõi kích thước của các gradient bằng TensorBoard), bạn có thể muốn thử cắt theo giá trị hoặc cắt theo chuẩn, với các ngưỡng khác nhau, và xem tùy chọn nào hoạt động tốt nhất trên tập xác thực.

Tái sử dụng các lớp đã được huấn luyện trước

Nói chung, không nên huấn luyện một DNN rất lớn từ đầu mà không thử tìm một mạng nơ-ron hiện có thực hiện một tác vụ tương tự với tác vụ bạn đang cố gắng giải quyết (tôi sẽ thảo luận cách tìm chúng trong Chương 14). Nếu bạn tìm thấy một mạng nơ-ron như vậy, thì bạn thường có thể tái sử dụng hầu hết các lớp của nó, ngoại trừ các lớp trên cùng. Kỹ thuật này được gọi là **học chuyển giao**. Nó sẽ không chỉ tăng tốc quá trình huấn luyện đáng kể, mà còn yêu cầu ít dữ liệu huấn luyện hơn đáng kể.

Giả sử bạn có quyền truy cập vào một DNN đã được huấn luyện để phân loại hình ảnh thành 100 danh mục khác nhau, bao gồm động vật, thực vật, phương tiện và các vật thể hàng ngày, và bây giờ bạn muốn huấn luyện một DNN để phân loại các loại phương tiện cụ thể. Các tác vụ này rất giống nhau, thậm chí một phần chồng chéo, vì vậy bạn nên thử tái sử dụng các phần của mạng đầu tiên (xem Hình 11-5).



Hình 11-5. Tái sử dụng các lớp đã được huấn luyện trước

Lớp đầu ra của mô hình gốc thường nên được thay thế vì nó rất có thể không hữu ích chút nào cho tác vụ mới, và có lẽ sẽ không có số lượng đầu ra phù hợp.

Tương tự, các lớp ẩn trên của mô hình gốc ít có khả năng hữu ích như các lớp thấp hơn, vì các đặc trưng cấp cao hữu ích nhất cho tác vụ mới có thể khác biệt đáng kể so với những đặc trưng hữu ích nhất cho tác vụ gốc. Bạn muốn tìm số lượng lớp phù hợp để tái sử dụng.

Hãy thử đóng băng tất cả các lớp đã được tái sử dụng trước (tức là làm cho trọng số của chúng không thể huấn luyện để giảm độ dốc sẽ không sửa đổi chúng và chúng sẽ vẫn cố định), sau đó huấn luyện mô hình của bạn và xem nó hoạt động như thế nào. Sau đó, thử bỏ đóng băng một hoặc hai lớp ẩn trên cùng để cho phép backpropagation điều chỉnh chúng và xem liệu hiệu suất có cải thiện không. Bạn càng có nhiều dữ liệu huấn luyện, bạn càng có thể bỏ đóng băng nhiều lớp. Cũng hữu ích khi giảm tốc độ học khi bạn bỏ đóng băng các lớp đã tái sử dụng: điều này sẽ tránh làm hỏng các trọng số đã được tinh chỉnh của chúng.

Nếu bạn vẫn không thể có được hiệu suất tốt, và bạn có ít dữ liệu huấn luyện, hãy thử bỏ lớp ẩn trên cùng và đóng băng tất cả các lớp ẩn còn lại một lần nữa. Bạn có thể lặp lại cho đến khi tìm thấy số lượng lớp phù hợp để tái sử dụng. Nếu bạn có nhiều dữ liệu huấn luyện, bạn có thể thử thay thế các lớp ẩn trên cùng thay vì bỏ chúng, và thậm chí thêm nhiều lớp ẩn hơn.

Học chuyển giao với Keras

Hãy xem một ví dụ. Giả sử tập dữ liệu Fashion MNIST chỉ chứa tám lớp — ví dụ, tất cả các lớp ngoại trừ sandal và shirt. Ai đó đã xây dựng và huấn luyện một mô hình Keras trên tập dữ liệu này và đạt được hiệu suất khá tốt (độ chính xác >90%). Gọi mô hình này là mô hình A. Bây giờ bạn muốn giải quyết một tác vụ khác: bạn có hình ảnh áo thun và áo chui đầu, và bạn muốn huấn luyện một bộ phân loại nhị phân: tích cực cho áo thun (và áo phông), tiêu cực cho sandal.

Tập dữ liệu của bạn khá nhỏ; bạn chỉ có 200 hình ảnh được gắn nhãn. Khi bạn huấn luyện một mô hình mới cho tác vụ này (gọi là mô hình B) với cùng kiến trúc như mô hình A, bạn đạt độ chính xác kiểm tra 91.85%. Trong khi uống cà phê buổi sáng, bạn nhận ra rằng tác vụ của mình khá giống với tác vụ A, vì vậy có lẽ học chuyển giao có thể giúp ích?

Hãy cùng tìm hiểu!

Đầu tiên, bạn cần tải mô hình A và tạo một mô hình mới dựa trên các lớp của mô hình đó. Bạn quyết định tái sử dụng tất cả các lớp ngoại trừ lớp đầu ra:

```
# ... Giả sử mô hình A đã được huấn luyện và lưu vào "my_model_A"
model_A = tf.keras.models.load_model("my_model_A")
model_B_on_A = tf.keras.Sequential(model_A.layers[:-1])
model_B_on_A.add(tf.keras.layers.Dense(1, activation="sigmoid"))
```

Lưu ý rằng `model_A` và `model_B_on_A` hiện chia sẻ một số lớp. Khi bạn huấn luyện `model_B_on_A`, nó cũng sẽ ảnh hưởng đến `model_A`. Nếu bạn muốn tránh điều đó, bạn cần sao chép `model_A` trước khi bạn tái sử dụng các lớp của nó. Để làm điều này, bạn sao chép kiến trúc của mô hình A bằng `clone_model()`, sau đó sao chép trọng số của nó:

```
model_A_clone = tf.keras.models.clone_model(model_A)
model_A_clone.set_weights(model_A.get_weights())
```

Bây giờ bạn có thể huấn luyện `model_B_on_A` cho tác vụ B, nhưng vì lớp đầu ra mới được khởi tạo ngẫu nhiên nên nó sẽ tạo ra các lỗi lớn (ít nhất là trong vài epoch đầu tiên), do đó sẽ có các gradient lỗi lớn có thể làm hỏng các trọng số đã được tái sử dụng. Để tránh điều này, một cách tiếp cận là đóng băng các lớp đã tái sử dụng trong vài epoch đầu tiên, cho phép lớp mới có thời gian để học các trọng số hợp lý. Để làm điều này, đặt thuộc tính `trainable` của mỗi lớp thành `False` và biên dịch mô hình:

```
for layer in model_B_on_A.layers[:-1]:
    layer.trainable = False

optimizer = tf.keras.optimizers.SGD(learning_rate=0.001)
model_B_on_A.compile(loss="binary_crossentropy", optimizer=optimizer,
                    metrics=["accuracy"])
```

Bây giờ bạn có thể huấn luyện mô hình trong vài epoch, sau đó bỏ đóng băng các lớp đã tái sử dụng (điều này yêu cầu biên dịch lại mô hình) và tiếp tục huấn luyện để tinh chỉnh các lớp

đã tái sử dụng cho tác vụ B. Sau khi bỏ đóng băng các lớp đã tái sử dụng, thường là một ý kiến hay để giảm tốc độ học, một lần nữa để tránh làm hỏng các trọng số đã tái sử dụng.

```
history = model_B_on_A.fit(X_train_B, y_train_B, epochs=4,
                           validation_data=(X_valid_B, y_valid_B))

for layer in model_B_on_A.layers[:-1]:
    layer.trainable = True

optimizer = tf.keras.optimizers.SGD(learning_rate=0.001)
model_B_on_A.compile(loss="binary_crossentropy", optimizer=optimizer,
                     metrics=["accuracy"])
history = model_B_on_A.fit(X_train_B, y_train_B, epochs=16,
                           validation_data=(X_valid_B, y_valid_B))
```

Vậy, phán quyết cuối cùng là gì? Chà, độ chính xác kiểm tra của mô hình này là 93.85%, tăng chính xác hai phần trăm từ 91.85%! Điều này có nghĩa là học chuyển giao đã giảm tỷ lệ lỗi gần 25%:

```
>>> model_B_on_A.evaluate(X_test_B, y_test_B)
[0.2546142041683197, 0.9384999871253967]
```

Bạn có bị thuyết phục không? Bạn không nên: tôi đã gian lận! Tôi đã thử nhiều cấu hình cho đến khi tôi tìm thấy một cấu hình thể hiện sự cải thiện mạnh mẽ. Nếu bạn thử thay đổi các lớp hoặc hạt giống ngẫu nhiên, bạn sẽ thấy rằng sự cải thiện nói chung giảm, hoặc thậm chí biến mất hoặc đảo ngược. Điều tôi đã làm được gọi là “tra tấn dữ liệu cho đến khi nó thú nhận”. Khi một bài báo trông quá tích cực, bạn nên nghi ngờ: có lẽ kỹ thuật mới lạ không thực sự giúp ích nhiều (trên thực tế, nó thậm chí có thể làm giảm hiệu suất), nhưng các tác giả đã thử nhiều biến thể và chỉ báo cáo kết quả tốt nhất (có thể do may mắn thuần túy), mà không đề cập đến số lần thất bại họ gặp phải trên đường đi. Hầu hết thời gian, điều này hoàn toàn không có ác ý, nhưng nó là một phần lý do tại sao rất nhiều kết quả trong khoa học không bao giờ có thể được tái tạo.

Tại sao tôi lại gian lận? Hóa ra học chuyển giao không hoạt động tốt lắm với các mạng dày đặc nhỏ, có lẽ vì các mạng nhỏ học ít mẫu, và các mạng dày đặc học các mẫu rất cụ thể, không có khả năng hữu ích trong các tác vụ khác. Học chuyển giao hoạt động tốt nhất với các mạng nơ-ron tích chập sâu, có xu hướng học các bộ phát hiện đặc trưng tổng quát hơn nhiều (đặc biệt là ở các lớp thấp hơn). Chúng ta sẽ xem xét lại học chuyển giao trong Chương 14, sử dụng các kỹ thuật chúng ta vừa thảo luận (và lần này sẽ không có gian lận, tôi hứa!).

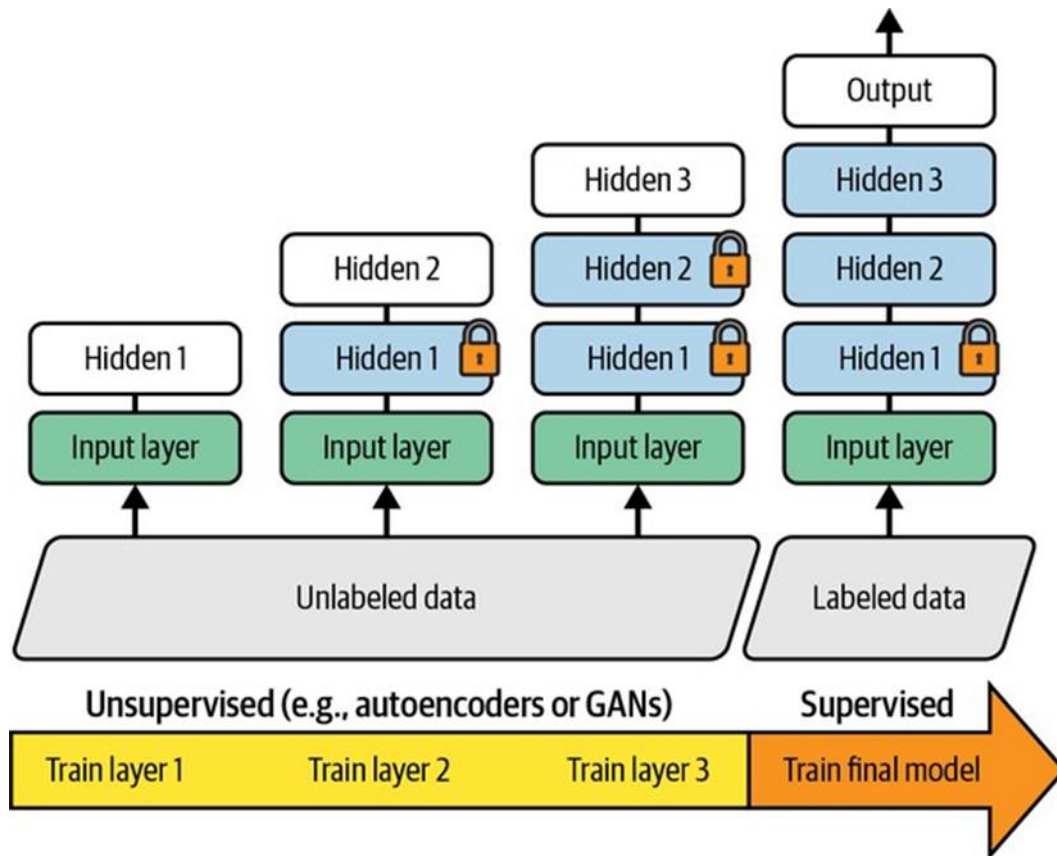
Tiền huấn luyện không giám sát

Giả sử bạn muốn giải quyết một tác vụ phức tạp mà bạn không có nhiều dữ liệu huấn luyện được gắn nhãn, nhưng thật không may, bạn không thể tìm thấy một mô hình đã được huấn luyện trên một tác vụ tương tự. Đừng mất hy vọng! Đầu tiên, bạn nên thử thu thập thêm dữ liệu huấn luyện được gắn nhãn, nhưng nếu không thể, bạn vẫn có thể thực hiện **tiền huấn luyện không giám sát** (xem Hình 11-6). Thật vậy, việc thu thập các ví dụ huấn luyện không gắn nhãn thường rẻ, nhưng việc gắn nhãn chúng thì đắt. Nếu bạn có thể thu thập nhiều dữ

liệu huấn luyện không gắn nhãn, bạn có thể thử sử dụng nó để huấn luyện một mô hình không giám sát, chẳng hạn như một bộ tự mã hóa (autoencoder) hoặc một mạng đối kháng tạo sinh (GAN; xem Chương 17). Sau đó, bạn có thể tái sử dụng các lớp thấp hơn của bộ tự mã hóa hoặc các lớp thấp hơn của bộ phân biệt của GAN, thêm lớp đầu ra cho tác vụ của bạn lên trên, và tinh chỉnh mạng cuối cùng bằng cách sử dụng học có giám sát (tức là với các ví dụ huấn luyện được gắn nhãn).

Chính kỹ thuật này đã được Geoffrey Hinton và nhóm của ông sử dụng vào năm 2006, và điều này đã dẫn đến sự hồi sinh của mạng nơ-ron và sự thành công của học sâu. Cho đến năm 2010, tiền huấn luyện không giám sát — thường là với các máy Boltzmann hạn chế (RBMs; xem sổ tay tại <https://homl.info/extra-anns>) — là tiêu chuẩn cho các mạng sâu, và chỉ sau khi vấn đề gradient biến mất được giảm nhẹ thì việc huấn luyện DNN hoàn toàn bằng học có giám sát mới trở nên phổ biến hơn nhiều. Tiền huấn luyện không giám sát (ngày nay thường sử dụng bộ tự mã hóa hoặc GANs thay vì RBMs) vẫn là một lựa chọn tốt khi bạn có một tác vụ phức tạp cần giải quyết, không có mô hình tương tự nào bạn có thể tái sử dụng, và ít dữ liệu huấn luyện được gắn nhãn nhưng có nhiều dữ liệu huấn luyện không gắn nhãn.

Lưu ý rằng trong những ngày đầu của học sâu, việc huấn luyện các mô hình sâu rất khó khăn, vì vậy mọi người sẽ sử dụng một kỹ thuật gọi là **tiền huấn luyện từng lớp tham lam** (greedy layer-wise pretraining) (được mô tả trong Hình 11-6). Họ sẽ huấn luyện một mô hình không giám sát với một lớp duy nhất, thường là một RBM, sau đó họ sẽ đóng băng lớp đó và thêm một lớp khác lên trên nó, sau đó huấn luyện lại mô hình (chỉ huấn luyện lớp mới một cách hiệu quả), sau đó đóng băng lớp mới và thêm một lớp khác lên trên nó, huấn luyện lại mô hình, v.v. Ngày nay, mọi thứ đơn giản hơn nhiều: mọi người thường huấn luyện mô hình không giám sát đầy đủ trong một lần và sử dụng bộ tự mã hóa hoặc GANs thay vì RBMs.



Hình 11-6. Trong huấn luyện không giám sát, một mô hình được huấn luyện trên tất cả dữ liệu, bao gồm dữ liệu không gắn nhãn, sử dụng kỹ thuật học không giám sát, sau đó nó được tinh chỉnh cho tác vụ cuối cùng chỉ trên dữ liệu được gắn nhãn bằng kỹ thuật học có giám sát; phần không giám sát có thể huấn luyện từng lớp một như thể hiện ở đây, hoặc nó có thể huấn luyện toàn bộ mô hình trực tiếp

Tiền huấn luyện (Pretraining) trên một Tác vụ Phụ trợ

Nếu bạn không có nhiều dữ liệu huấn luyện có nhãn, một lựa chọn cuối cùng là **huấn luyện một mạng nơ-ron đầu tiên trên một tác vụ phụ trợ** mà bạn có thể dễ dàng thu thập hoặc tạo ra dữ liệu huấn luyện có nhãn, sau đó **tái sử dụng các tầng dưới** của mạng đó cho tác vụ thực tế của bạn. Các tầng dưới của mạng nơ-ron đầu tiên sẽ học được các bộ phát hiện đặc trưng (feature detectors) mà có khả năng tái sử dụng được bởi mạng nơ-ron thứ hai.

Ví dụ, nếu bạn muốn xây dựng một hệ thống để nhận dạng khuôn mặt, bạn có thể chỉ có một vài bức ảnh của mỗi cá nhân—rõ ràng là không đủ để huấn luyện một bộ phân loại tốt. Việc thu thập hàng trăm bức ảnh của mỗi người sẽ không thực tế. Tuy nhiên, bạn có thể thu thập rất nhiều ảnh của những người ngẫu nhiên trên web và huấn luyện một mạng nơ-ron đầu tiên để phát hiện xem hai bức ảnh khác nhau có phải là cùng một người hay không. Một mạng như vậy sẽ học được các bộ phát hiện đặc trưng tốt cho khuôn mặt, vì vậy việc tái sử dụng

các tầng dưới của nó sẽ cho phép bạn huấn luyện một bộ phân loại khuôn mặt tốt chỉ với ít dữ liệu huấn luyện.

Đối với các ứng dụng xử lý ngôn ngữ tự nhiên (NLP), bạn có thể tải xuống một kho văn bản (corpus) gồm hàng triệu tài liệu và tự động tạo dữ liệu có nhãn từ đó. Ví dụ, bạn có thể che ngẫu nhiên một số từ và huấn luyện một mô hình để dự đoán các từ còn thiếu là gì (ví dụ, nó nên dự đoán rằng từ còn thiếu trong câu “What _____ you saying?” có lẽ là “are” hoặc “were”). Nếu bạn có thể huấn luyện một mô hình để đạt được hiệu suất tốt trong tác vụ này, thì nó sẽ biết khá nhiều về ngôn ngữ, và bạn chắc chắn có thể tái sử dụng nó cho tác vụ thực tế của mình và tinh chỉnh (fine-tune) nó trên dữ liệu có nhãn của bạn (chúng ta sẽ thảo luận thêm về các tác vụ tiền huấn luyện trong Chương 15).

Các Trình tối ưu hóa Nhanh hơn

Huấn luyện một mạng nơ-ron sâu rất lớn có thể chậm một cách đau đớn. Cho đến nay, chúng ta đã thấy bốn cách để tăng tốc độ huấn luyện (và đạt được một giải pháp tốt hơn): áp dụng một chiến lược **khởi tạo** tốt cho các trọng số kết nối, sử dụng một **hàm kích hoạt** tốt, sử dụng **chuẩn hóa theo lô** (batch normalization), và **tái sử dụng các phần của một mạng đã được tiền huấn luyện** (có thể được xây dựng cho một tác vụ phụ trợ hoặc sử dụng học không giám sát). Một sự tăng tốc lớn khác đến từ việc sử dụng một trình tối ưu hóa nhanh hơn so với trình tối ưu hóa hạ gradient thông thường. Trong phần này, chúng ta sẽ trình bày các thuật toán tối ưu hóa phổ biến nhất: **momentum**, **Nesterov accelerated gradient**, **AdaGrad**, **RMSProp**, và cuối cùng là **Adam** và các biến thể của nó.

Momentum (Động lượng)

Hãy tưởng tượng một quả bóng bowling lăn xuống một con dốc thoải thoải trên một bề mặt nhẵn: nó sẽ bắt đầu chậm, nhưng sẽ nhanh chóng tăng tốc cho đến khi đạt đến vận tốc cuối (terminal velocity) (nếu có một số ma sát hoặc sức cản không khí). Đây là ý tưởng cốt lõi đằng sau **tối ưu hóa động lượng**, được đề xuất bởi Boris Polyak vào năm 1964. Ngược lại, hạ gradient thông thường sẽ thực hiện các bước nhỏ khi dốc thoải và các bước lớn khi dốc đứng, nhưng nó sẽ không bao giờ tăng tốc. Do đó, hạ gradient thông thường thường chậm hơn nhiều trong việc đạt đến điểm cực tiểu so với tối ưu hóa động lượng.

Hãy nhớ lại rằng hạ gradient cập nhật trọng số θ bằng cách trừ đi gradient của hàm chi phí $J(\theta)$ đối với trọng số, được nhân với tốc độ học η . Phương trình là $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$. Nó không quan tâm đến các gradient trước đó. Nếu gradient cục bộ nhỏ, nó sẽ di chuyển rất chậm.

Hạ gradient động lượng quan tâm rất nhiều đến các gradient trước đó: ở mỗi lần lặp, nó trừ gradient cục bộ khỏi vector động lượng m (đã được nhân với tốc độ học η), và cập nhật trọng số bằng cách thêm vector động lượng này. Nói cách khác, gradient được sử dụng như một gia tốc, chứ không phải là một tốc độ. Để mô phỏng một cơ chế ma sát nào đó và ngăn động

lượng phát triển quá lớn, thuật toán giới thiệu một siêu tham số mới β , được gọi là **động lượng**. Giá trị của β phải nằm giữa 0 (không có ma sát) và 1. Một giá trị động lượng điển hình là 0,9.

Công thức 11-5: Thuật toán động lượng

- $m \leftarrow \beta m - \eta \nabla_{\theta} J(\theta)$
- $\theta \leftarrow \theta + m$

Bạn có thể xác minh rằng nếu gradient không đổi, vận tốc cuối cùng (tức là kích thước cập nhật trọng số tối đa) bằng với gradient đó nhân với tốc độ học nhân với $1/(1 - \beta)$ (bỏ qua dấu). Ví dụ: nếu $\beta = 0.9$, thì vận tốc cuối cùng bằng 10 lần gradient nhân với tốc độ học, vì vậy hạ gradient động lượng cuối cùng đi nhanh gấp 10 lần so với hạ gradient thông thường. Điều này cho phép hạ gradient động lượng thoát ra khỏi các vùng bình nguyên (plateaus) nhanh hơn nhiều so với hạ gradient thông thường. Chúng ta đã thấy trong Chương 4 rằng khi các đầu vào có các tỷ lệ khác nhau, hàm chi phí sẽ có hình dạng giống như một cái bát thuôn dài. Hạ gradient thông thường đi xuống dốc khá nhanh, nhưng sau đó phải mất một thời gian rất dài để đi xuống thung lũng. Ngược lại, hạ gradient động lượng sẽ lăn xuống thung lũng nhanh hơn và dễ dàng hơn cho đến khi nó đến đáy (cực tiểu). Trong các mạng nơ-ron sâu nơi các đầu vào không có cùng tỷ lệ, các lớp trên sẽ thường kết thúc với các đầu vào có các tỷ lệ rất khác nhau, vì vậy việc sử dụng hạ gradient động lượng giúp rất nhiều. Nó cũng có thể giúp vượt qua các cực tiểu cục bộ.

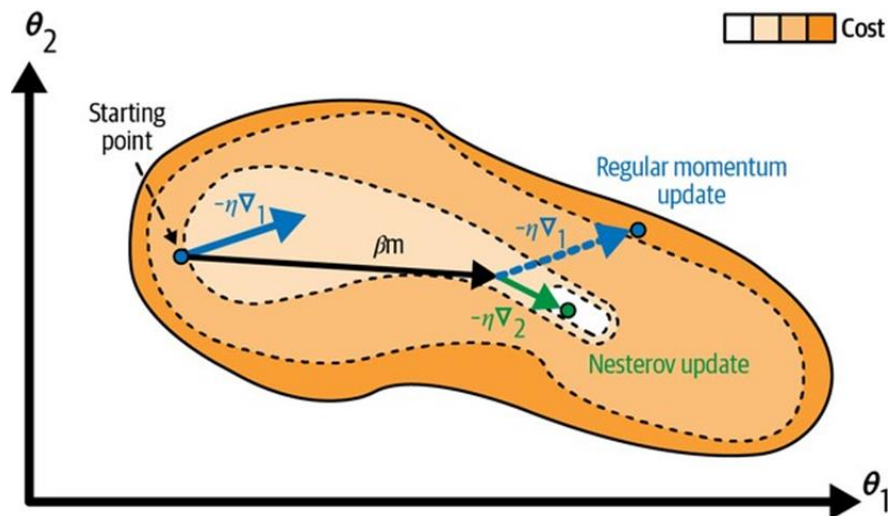
Nesterov Accelerated Gradient (NAG)

Một biến thể nhỏ của tối ưu hóa động lượng, do Yurii Nesterov đề xuất năm 1983, gần như luôn nhanh hơn tối ưu hóa động lượng thông thường. Phương pháp **Nesterov accelerated gradient (NAG)**, còn được gọi là tối ưu hóa động lượng Nesterov, đo gradient của hàm chi phí không phải tại vị trí cục bộ θ mà là **hơi lệch về phía trước theo hướng của động lượng**, tại $\theta + \beta m$ (xem Phương trình 11-6).

Phương trình 11-6. Thuật toán Nesterov Accelerated Gradient

- $m \leftarrow \beta m - \eta \nabla_{\theta} J(\theta + \beta m)$
- $\theta \leftarrow \theta + m$

Sự điều chỉnh nhỏ này hoạt động vì nói chung vector động lượng sẽ trở đúng hướng (tức là về phía điểm tối ưu), vì vậy sẽ chính xác hơn một chút khi sử dụng gradient được đo ở vị trí xa hơn một chút theo hướng đó thay vì gradient tại vị trí ban đầu.



Hình 11-7. So sánh tối ưu hóa bằng động lượng thông thường và động lượng Nesterov: phương pháp đầu tiên áp dụng gradient được tính trước bước động lượng, trong khi phương pháp thứ hai áp dụng gradient được tính sau.

Như bạn có thể thấy, cập nhật Nesterov kết thúc gần điểm tối ưu hơn. Sau một thời gian, những cải tiến nhỏ này cộng lại và NAG cuối cùng nhanh hơn đáng kể so với tối ưu hóa động lượng thông thường. Hơn nữa, lưu ý rằng khi động lượng đẩy các trọng số qua một thung lũng, ∇_1 tiếp tục đẩy xa hơn qua thung lũng, trong khi ∇_2 đẩy ngược lại về phía đáy thung lũng. Điều này giúp giảm dao động và do đó NAG hội tụ nhanh hơn.

Để sử dụng NAG, chỉ cần đặt `nesterov=True` khi tạo trình tối ưu hóa SGD:

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.001, momentum=0.9, nesterov=True)
```

AdaGrad

Hãy xem xét lại vấn đề “cái bát dài”: hạ gradient bắt đầu bằng cách đi xuống nhanh chóng theo con dốc dốc nhất, vốn không chỉ thẳng đến điểm tối ưu toàn cục, sau đó nó đi xuống đáy thung lũng rất chậm. Sẽ thật tuyệt nếu thuật toán có thể **chỉnh hướng sớm hơn** để chỉ thẳng hơn về phía điểm tối ưu toàn cục. Thuật toán **AdaGrad** đạt được sự điều chỉnh này bằng cách **giảm tỷ lệ vector gradient dọc theo các chiều dốc nhất** (xem Phương trình 11-7).

Công thức 11-7: Thuật toán AdaGrad

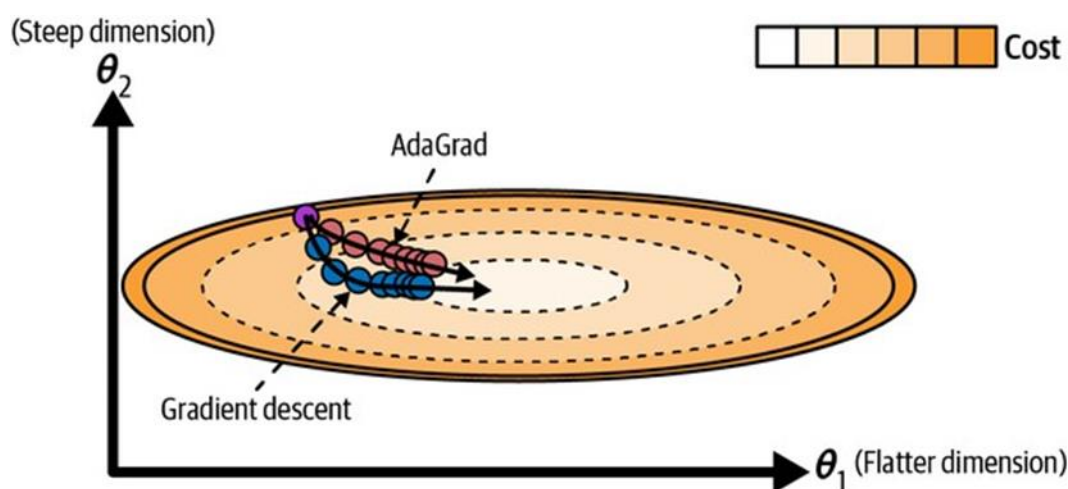
- $s \leftarrow s + \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$
- $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta) \oslash \sqrt{s + \epsilon}$

Bước đầu tiên tích lũy bình phương của các gradient vào vector s (ký hiệu $\otimes$ đại diện cho phép nhân từng phần tử). Dạng vector này tương đương với việc tính toán $s_i \leftarrow s_i +$

$(\partial J(\theta) / \partial \theta_i)^2$ cho mỗi phần tử s_i của vector s . Nếu hàm chi phí dốc dọc theo chiều thứ i , s_i sẽ trở nên lớn hơn sau mỗi lần lặp.

Bước thứ hai gần như giống hệt hạ gradient, nhưng có một khác biệt lớn: vector gradient được chia tỷ lệ bởi một hệ số $\sqrt{s_i + \epsilon}$ ($\oslash$ là ký hiệu cho phép chia từng phần tử, và ϵ là một số hạng làm mịn để tránh chia cho 0, thường được đặt là 10^{-10}). Dạng vector này tương đương với việc tính toán đồng thời $\theta_i \leftarrow \theta_i - \eta \partial J(\theta) / \partial \theta_i / \sqrt{s_i + \epsilon}$ cho tất cả các tham số.

Nói tóm lại, thuật toán này làm giảm tốc độ học, nhưng nó làm như vậy nhanh hơn đối với các chiều dốc hơn so với các chiều có độ dốc nhẹ nhàng hơn. Điều này được gọi là **tốc độ học thích ứng**. Nó giúp các bản cập nhật hướng trực tiếp hơn đến cực tiểu toàn cục. Một lợi ích bổ sung là nó đòi hỏi ít điều chỉnh hơn đối với siêu tham số tốc độ học η .



Hình 11-8. So sánh AdaGrad và gradient descent: phương pháp đầu tiên (AdaGrad) có thể điều chỉnh hướng đi sớm hơn để hướng về điểm tối ưu.

AdaGrad thường hoạt động tốt cho các bài toán bậc hai đơn giản, nhưng nó thường dừng lại quá sớm khi huấn luyện mạng nơ-ron. Vì vậy, mặc dù Keras có trình tối ưu hóa Adagrad, bạn không nên sử dụng nó để huấn luyện các mạng nơ-ron sâu.

RMSProp

Như chúng ta đã thấy, AdaGrad có nguy cơ làm chậm tốc độ học quá nhanh và không bao giờ hội tụ về cực tiểu toàn cục. Thuật toán **RMSProp** khắc phục điều này bằng cách chỉ tích lũy gradient từ các lần lặp gần đây nhất, thay vì tất cả các gradient kể từ khi bắt đầu huấn luyện. Nó thực hiện điều này bằng cách sử dụng phép giảm theo cấp số nhân trong bước đầu tiên.

Công thức 11-8: Thuật toán RMSProp

- $s \leftarrow \rho s + (1 - \rho) \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$
- $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta) \oslash \sqrt{s + \epsilon}$

Tỷ lệ giảm ρ thường được đặt là 0.9. Vâng, nó lại là một siêu tham số mới, nhưng giá trị mặc định này thường hoạt động tốt, vì vậy bạn có thể không cần phải điều chỉnh nó.

Như bạn có thể mong đợi, Keras có một bộ tối ưu hóa RMSProp.

```
optimizer = tf.keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9)
```

Ngoại trừ các bài toán rất đơn giản, trình tối ưu hóa này gần như luôn hoạt động tốt hơn nhiều so với AdaGrad.

Adam

Adam, viết tắt của **adaptive moment estimation**, kết hợp các ý tưởng của tối ưu hóa động lượng và RMSProp: giống như tối ưu hóa động lượng, nó theo dõi trung bình suy giảm theo cấp số nhân của các gradient trong quá khứ; và giống như RMSProp, nó theo dõi trung bình suy giảm theo cấp số nhân của các bình phương gradient trong quá khứ.

Công thức 11-9: Thuật toán Adam

- $m \leftarrow \beta_1 m - (1 - \beta_1) \nabla_{\theta} J(\theta)$
- $s \leftarrow \beta_2 s + (1 - \beta_2) \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$
- $m \leftarrow \frac{m}{1 - \beta_1^t}$
- $s \leftarrow \frac{s}{1 - \beta_2^t}$
- $\theta \leftarrow \theta + \eta m \oslash \sqrt{s + \epsilon}$

Trong phương trình này, t là số lần lặp (bắt đầu từ 1).

Nếu bạn chỉ nhìn vào các bước 1, 2 và 5, bạn sẽ thấy sự tương đồng gần gũi của Adam với cả hạ gradient động lượng (momentum optimization) và RMSProp. β_1 tương ứng với động lượng, và β_2 tương ứng với ρ trong RMSProp. Điểm khác biệt duy nhất là bước 1 tính trung bình giảm theo cấp số nhân thay vì tổng giảm theo cấp số nhân, nhưng chúng thực sự tương đương nhau ngoại trừ một hằng số.

Các bước 3 và 4 là một chi tiết kỹ thuật: vì m và s được khởi tạo bằng 0 khi bắt đầu huấn luyện, nên chúng sẽ bị lệch về 0. Do đó, hai bước này sẽ giúp tăng cường m và s khi bắt đầu huấn luyện.

Siêu tham số giảm động lượng β_1 thường được khởi tạo là 0.9, trong khi siêu tham số giảm tỷ lệ β_2 thường được khởi tạo là 0.999. Tương tự, số hạng làm mịn ϵ thường được khởi tạo là một số rất nhỏ như 10^{-7} . Đây là các giá trị mặc định cho lớp Adam.

```
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001, beta_1=0.9, beta_2=0.999)
```

Vì Adam là một thuật toán tốc độ học thích ứng, nó đòi hỏi ít tinh chỉnh hơn cho siêu tham số tốc độ học η .

Cuối cùng, ba biến thể của Adam đáng được đề cập: **AdaMax**, **Nadam**, và **AdamW**.

- **AdaMax**: Thay thế chuẩn ℓ_2 bằng chuẩn ℓ_∞ . Điều này có thể làm cho AdaMax ổn định hơn Adam, nhưng nói chung Adam hoạt động tốt hơn.
- **Nadam**: Là sự kết hợp giữa tối ưu hóa Adam và kỹ thuật Nesterov, vì vậy nó thường hội tụ nhanh hơn một chút so với Adam.
- **AdamW**: Là một biến thể của Adam tích hợp một kỹ thuật điều chuẩn (regularization) gọi là **suy giảm trọng số (weight decay)**. AdamW khắc phục vấn đề khi kết hợp Adam với điều chuẩn ℓ_2 , thường dẫn đến các mô hình tổng quát hóa không tốt bằng SGD.

Để sử dụng Nadam, AdaMax hoặc AdamW trong Keras, hãy thay thế `tf.keras.optimizers.Adam` bằng `tf.keras.optimizers.Nadam`, `tf.keras.optimizers.Adamax`, hoặc `tf.keras.optimizers.experimental.AdamW`.

Tất cả các kỹ thuật tối ưu hóa được thảo luận cho đến nay chỉ dựa vào đạo hàm riêng bậc nhất. Các thuật toán dựa trên đạo hàm riêng bậc hai (ma trận Hessian) rất khó áp dụng cho các mạng nơ-ron sâu vì chi phí tính toán và bộ nhớ rất lớn.

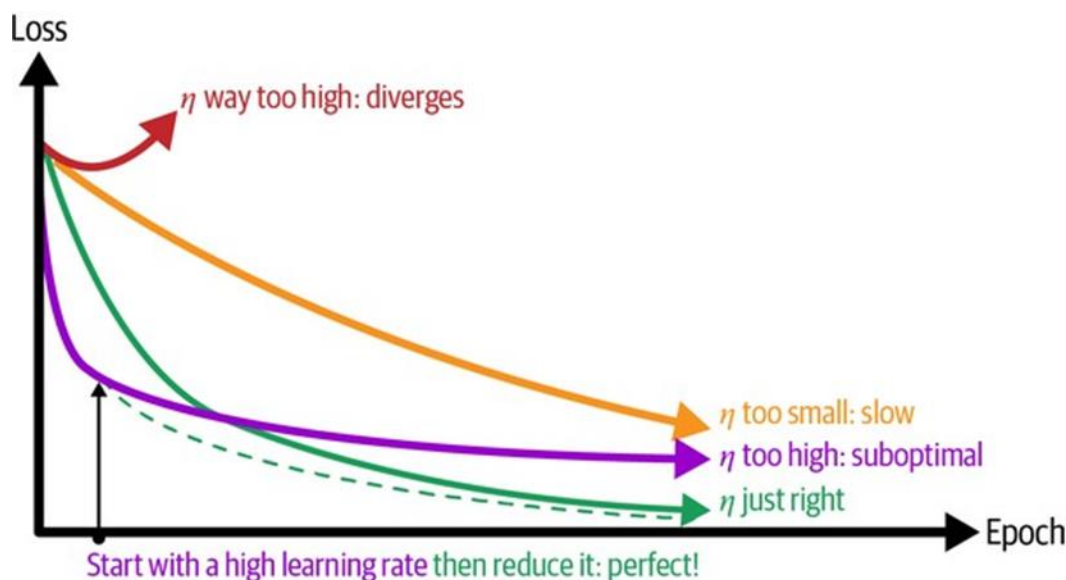
Bảng 11-2. So sánh các trình tối ưu hóa (* là kém, ** là trung bình, và *** là tốt).

Lớp	Tốc độ hội tụ	Chất lượng hội tụ
SGD	*	***
SGD(momentum=...)	**	***
SGD(momentum=..., nesterov=True)	**	***
Adagrad	***	* (dừng quá sớm)
RMSprop	***	** hoặc ***
Adam	***	** hoặc ***
AdaMax	***	** hoặc ***
Nadam	***	** hoặc ***
AdamW	***	** hoặc ***

Lập lịch tốc độ học (Learning Rate Scheduling)

Việc tìm một tốc độ học tốt là rất quan trọng. Nếu bạn đặt nó quá cao, quá trình huấn luyện có thể phân kỳ (như đã thảo luận trong phần “Gradient Descent”). Nếu bạn đặt nó quá thấp, quá trình huấn luyện cuối cùng sẽ hội tụ đến cực tối ưu, nhưng sẽ mất rất nhiều thời gian. Nếu bạn đặt nó hơi quá cao, nó sẽ tiến bộ rất nhanh lúc đầu, nhưng nó sẽ kết thúc việc nhảy

múa quanh cực tối ưu và không bao giờ thực sự ổn định. Nếu bạn có ngân sách tính toán hạn chế, bạn có thể phải gián đoạn huấn luyện trước khi nó hội tụ đúng cách, dẫn đến một giải pháp không tối ưu (xem Hình 11-9).



Hình 11-9. Các đường cong học tập cho các tốc độ học khác nhau η

Như đã thảo luận trong Chương 10, bạn có thể tìm thấy một tốc độ học tốt bằng cách huấn luyện mô hình trong vài trăm lần lặp, tăng tốc độ học theo cấp số mũ từ một giá trị rất nhỏ đến một giá trị rất lớn, và sau đó nhìn vào đường cong học tập và chọn một tốc độ học thấp hơn một chút so với điểm mà đường cong học tập bắt đầu tăng vọt trở lại. Sau đó, bạn có thể khởi tạo lại mô hình của mình và huấn luyện nó với tốc độ học đó.

Nhưng bạn có thể làm tốt hơn một tốc độ học cố định: nếu bạn bắt đầu với một tốc độ học lớn và sau đó giảm nó khi quá trình huấn luyện ngừng tiến triển nhanh, bạn có thể đạt được một giải pháp tốt nhanh hơn so với tốc độ học cố định tối ưu. Có nhiều chiến lược khác nhau để giảm tốc độ học trong quá trình huấn luyện. Cũng có thể có lợi khi bắt đầu với tốc độ học thấp, tăng nó, sau đó giảm nó trở lại. Các chiến lược này được gọi là **lịch trình học tập** (tôi đã giới thiệu ngắn gọn khái niệm này trong Chương 4).

Đây là những lịch trình học tập được sử dụng phổ biến nhất:

- **Lập lịch theo hàm lũy thừa (Power scheduling):** Đặt tốc độ học là một hàm của số lần lặp t : $\eta(t) = \eta_0 / (1 + t/s)^c$. Tốc độ học ban đầu η_0 , số mũ c (thường được đặt là 1), và các bước s là các siêu tham số. Tốc độ học giảm ở mỗi bước. Sau s bước, tốc độ học giảm xuống còn $\eta_0/2$. Sau s bước nữa, nó giảm xuống còn $\eta_0/3$, sau đó nó giảm xuống còn $\eta_0/4$, sau đó $\eta_0/5$, v.v. Như bạn có thể thấy, lịch trình này ban đầu giảm nhanh, sau đó ngày càng chậm hơn. Tất nhiên, lập lịch theo hàm lũy thừa yêu cầu điều chỉnh η_0 và s (và có thể cả c).
- **Lập lịch theo hàm mũ (Exponential scheduling):** Đặt tốc độ học là $\eta(t) = \eta_0 \cdot 0.1^{t/s}$. Tốc độ học sẽ dần dần giảm đi một hệ số 10 sau mỗi s bước. Trong

khi lập lịch theo hàm lũy thừa làm giảm tốc độ học ngày càng chậm hơn, lập lịch theo hàm mũ vẫn tiếp tục cắt giảm nó theo hệ số 10 sau mỗi s bước.

- **Lập lịch hằng số từng phần (Piecewise constant scheduling):** Sử dụng tốc độ học không đổi trong một số epoch (ví dụ: $\eta_{a_0} = 0.1$ trong 5 epoch), sau đó là tốc độ học nhỏ hơn cho một số epoch khác (ví dụ: $\eta_{a_1} = 0.001$ trong 50 epoch), và cứ thế. Mặc dù giải pháp này có thể hoạt động rất tốt, nhưng nó đòi hỏi phải điều chỉnh để tìm ra chuỗi tốc độ học phù hợp và thời gian sử dụng mỗi tốc độ.
- **Lập lịch theo hiệu suất (Performance scheduling):** Đo lỗi xác thực sau mỗi N bước (giống như dừng sớm), và giảm tốc độ học bằng một hệ số λ khi lỗi ngừng giảm.
- **Lập lịch 1 chu kỳ (1cycle scheduling):** 1cycle được giới thiệu trong một bài báo năm 2018 của Leslie Smith. Trái ngược với các cách tiếp cận khác, nó bắt đầu bằng cách tăng tốc độ học ban đầu η_{a_0} , tăng tuyến tính lên đến η_{a_1} ở giữa quá trình huấn luyện. Sau đó, nó giảm tốc độ học tuyến tính xuống η_{a_0} một lần nữa trong nửa sau của quá trình huấn luyện, kết thúc vài epoch cuối cùng bằng cách giảm tốc độ xuống vài bậc độ lớn (vẫn tuyến tính). Tốc độ học tối đa η_{a_1} được chọn bằng cách sử dụng cùng một cách tiếp cận mà chúng ta đã sử dụng để tìm tốc độ học tối ưu, và tốc độ học ban đầu η_{a_0} thường thấp hơn 10 lần. Khi sử dụng đà, chúng ta bắt đầu với đà cao trước (ví dụ: 0.95), sau đó giảm nó xuống đà thấp hơn trong nửa đầu của quá trình huấn luyện (ví dụ: xuống 0.85, tuyến tính), và sau đó đưa nó trở lại giá trị tối đa (ví dụ: 0.95) trong nửa sau của quá trình huấn luyện, kết thúc vài epoch cuối cùng với giá trị tối đa đó. Smith đã thực hiện nhiều thí nghiệm cho thấy cách tiếp cận này thường có thể tăng tốc đáng kể quá trình huấn luyện và đạt được hiệu suất tốt hơn. Ví dụ, trên tập dữ liệu hình ảnh CIFAR10 phổ biến, cách tiếp cận này đạt độ chính xác xác thực 91.9% chỉ trong 100 epoch, so với độ chính xác 90.3% trong 800 epoch thông qua một cách tiếp cận tiêu chuẩn (với cùng kiến trúc mạng nơ-ron). Thành tựu này được mệnh danh là siêu hội tụ (super-convergence).

Một bài báo năm 2013 của Andrew Senior và cộng sự đã so sánh hiệu suất của một số lịch trình học tập phổ biến nhất khi sử dụng tối ưu hóa đà để huấn luyện mạng nơ-ron sâu cho nhận dạng giọng nói. Các tác giả kết luận rằng, trong cài đặt này, cả lập lịch theo hiệu suất và lập lịch theo hàm mũ đều hoạt động tốt. Họ ủng hộ lập lịch theo hàm mũ vì nó dễ điều chỉnh và nó hội tụ nhanh hơn một chút đến giải pháp tối ưu. Họ cũng đề cập rằng nó dễ thực hiện hơn lập lịch theo hiệu suất, nhưng trong Keras cả hai tùy chọn đều dễ dàng. Điều đó nói lên rằng, cách tiếp cận 1cycle dường như hoạt động tốt hơn nữa.

Triển khai lập lịch theo hàm lũy thừa trong Keras là lựa chọn dễ nhất - chỉ cần đặt siêu tham số decay khi tạo một bộ tối ưu hóa:

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01, decay=1e-4)
```

decay là nghịch đảo của s (số bước để chia tốc độ học thêm một đơn vị), và Keras giả định rằng c bằng 1.

Lập lịch theo hàm mũ và lập lịch từng phần cũng khá đơn giản. Bạn cần định nghĩa một hàm nhận epoch hiện tại và trả về tốc độ học. Ví dụ, hãy triển khai lập lịch theo hàm mũ:

```
def exponential_decay_fn(epoch):
    return 0.01 * 0.1 ** (epoch / 20)
```

Nếu bạn không muốn mã hóa cứng eta_0 và s , bạn có thể tạo một hàm trả về một hàm đã được cấu hình:

```
def exponential_decay(lr0, s):
    def exponential_decay_fn(epoch):
        return lr0 * 0.1 ** (epoch / s)
    return exponential_decay_fn
```

```
exponential_decay_fn = exponential_decay(lr0=0.01, s=20)
```

Tiếp theo, tạo một callback `LearningRateScheduler`, truyền cho nó hàm lịch trình, và truyền callback này vào phương thức `fit()`:

```
lr_scheduler = tf.keras.callbacks.LearningRateScheduler(exponential_decay_fn)
history = model.fit(X_train, y_train, [...], callbacks=[lr_scheduler])
```

`LearningRateScheduler` sẽ cập nhật thuộc tính `learning_rate` của bộ tối ưu hóa vào đầu mỗi epoch. Cập nhật tốc độ học một lần mỗi epoch thường là đủ, nhưng nếu bạn muốn nó được cập nhật thường xuyên hơn, ví dụ như ở mỗi bước, bạn luôn có thể viết callback của riêng mình (xem phần “Lập lịch theo hàm mũ” của sổ tay chương này để biết ví dụ). Cập nhật tốc độ học ở mỗi bước có thể hữu ích nếu có nhiều bước mỗi epoch. Ngoài ra, bạn có thể sử dụng cách tiếp cận `tf.keras.optimizers.schedules`, được mô tả ngay sau đây.

Hàm lịch trình tùy chọn có thể nhận tốc độ học hiện tại làm đối số thứ hai. Ví dụ, hàm lịch trình sau đây nhân tốc độ học trước đó với $0.1^{1/20}$, điều này dẫn đến cùng một sự suy giảm theo hàm mũ (ngoại trừ sự suy giảm bây giờ bắt đầu vào đầu epoch 0 thay vì 1):

```
def exponential_decay_fn(epoch, lr):
    return lr * 0.1 ** (1 / 20)
```

Việc triển khai này dựa vào tốc độ học ban đầu của bộ tối ưu hóa (trái ngược với triển khai trước), vì vậy hãy đảm bảo đặt nó một cách thích hợp.

Khi bạn lưu một mô hình, bộ tối ưu hóa và tốc độ học của nó cũng được lưu cùng với nó. Điều này có nghĩa là với hàm lịch trình mới này, bạn có thể chỉ cần tải một mô hình đã huấn luyện và tiếp tục huấn luyện từ nơi nó đã dừng lại, không có vấn đề gì. Tuy nhiên, mọi thứ không đơn giản như vậy nếu hàm lịch trình của bạn sử dụng đối số epoch: epoch không được lưu, và nó được đặt lại về 0 mỗi khi bạn gọi phương thức `fit()`. Nếu bạn tiếp tục huấn luyện một mô hình từ nơi nó đã dừng lại, điều này có thể dẫn đến tốc độ học rất lớn, điều này có thể làm hỏng trọng số của mô hình của bạn. Một giải pháp là đặt thủ công đối số `initial_epoch` của phương thức `fit()` để epoch bắt đầu ở giá trị đúng.

Đối với lập lịch hằng số từng phần, bạn có thể sử dụng một hàm lịch trình như sau (như trước, bạn có thể định nghĩa một hàm tổng quát hơn nếu muốn; xem phần “Lập lịch hằng số từng phần” của sổ tay để biết ví dụ), sau đó tạo một callback `LearningRateScheduler` với hàm này và truyền nó vào phương thức `fit()`, giống như lập lịch theo hàm mũ:

```
def piecewise_constant_fn(epoch):
    if epoch < 5:
        return 0.01
    elif epoch < 15:
        return 0.005
    else:
        return 0.001
```

Để lập lịch theo hiệu suất, hãy sử dụng callback `ReduceLROnPlateau`. Ví dụ, nếu bạn truyền callback sau đây vào phương thức `fit()`, nó sẽ nhân tốc độ học với 0.5 bất cứ khi nào mất mát xác thực tốt nhất không cải thiện trong năm epoch liên tiếp (các tùy chọn khác có sẵn; vui lòng kiểm tra tài liệu để biết thêm chi tiết):

```
lr_scheduler = tf.keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=5)
history = model.fit(X_train, y_train, [...], callbacks=[lr_scheduler])
```

Cuối cùng, Keras cung cấp một cách thay thế để triển khai lập lịch tốc độ học: bạn có thể định nghĩa một tốc độ học theo lịch trình bằng cách sử dụng một trong các lớp có sẵn trong `tf.keras.optimizers.schedules`, sau đó truyền nó cho bất kỳ bộ tối ưu hóa nào. Cách tiếp cận này cập nhật tốc độ học ở mỗi bước thay vì ở mỗi epoch. Ví dụ, đây là cách triển khai cùng một lịch trình theo hàm mũ như hàm `exponential_decay_fn()` chúng ta đã định nghĩa trước đó:

```
import math

batch_size = 32
n_epochs = 25
n_steps = n_epochs * math.ceil(len(X_train) / batch_size)
scheduled_learning_rate = tf.keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate=0.01, decay_steps=n_steps, decay_rate=0.1
)
optimizer = tf.keras.optimizers.SGD(learning_rate=scheduled_learning_rate)
```

Điều này tốt và đơn giản, hơn nữa khi bạn lưu mô hình, tốc độ học và lịch trình của nó (bao gồm trạng thái của nó) cũng được lưu.

Đối với 1 cycle, Keras không hỗ trợ nó, nhưng có thể triển khai nó trong chưa đầy 30 dòng mã bằng cách tạo một callback tùy chỉnh sửa đổi tốc độ học ở mỗi lần lặp. Để cập nhật tốc độ học của bộ tối ưu hóa từ bên trong phương thức `on_batch_begin()` của callback, bạn cần gọi `tf.keras.backend.set_value(self.model.optimizer.learning_rate, new_learning_rate)`. Xem phần “Lập lịch 1 chu kỳ” của sổ tay để biết ví dụ.

Tóm lại, sự suy giảm theo hàm mũ, lập lịch theo hiệu suất và 1 cycle có thể tăng tốc đáng kể sự hội tụ, vì vậy hãy thử chúng!

Tránh Overfitting thông qua chính quy hóa

“Với bốn tham số, tôi có thể lắp vừa một con voi và với năm tham số, tôi có thể làm cho nó vẩy vòi của mình.” —John von Neumann, được Enrico Fermi trích dẫn trong Nature 427

Với hàng nghìn tham số, bạn có thể lấp vừa cả vườn bách thú. Mạng nơ-ron sâu thường có hàng chục nghìn tham số, đôi khi thậm chí hàng triệu. Điều này mang lại cho chúng một lượng tự do đáng kinh ngạc và có nghĩa là chúng có thể phù hợp với rất nhiều tập dữ liệu phức tạp khác nhau. Nhưng sự linh hoạt tuyệt vời này cũng khiến mạng dễ bị overfitting tập huấn luyện. Chính quy hóa thường cần thiết để ngăn chặn điều này.

Chúng ta đã triển khai một trong những kỹ thuật chính quy hóa tốt nhất trong Chương 10: dùng sớm. Hơn nữa, mặc dù chuẩn hóa theo batch được thiết kế để giải quyết các vấn đề gradient không ổn định, nó cũng hoạt động như một bộ chính quy hóa khá tốt. Trong phần này, chúng ta sẽ xem xét các kỹ thuật chính quy hóa phổ biến khác cho mạng nơ-ron: chính quy hóa l_1 và l_2 , dropout, và chính quy hóa max-norm.

Chính quy hóa l_1 và l_2

Giống như bạn đã làm trong Chương 4 đối với các mô hình tuyến tính đơn giản, bạn có thể sử dụng chính quy hóa l_2 để ràng buộc các trọng số kết nối của mạng nơ-ron, và/hoặc chính quy hóa l_1 nếu bạn muốn một mô hình thưa thớt (với nhiều trọng số bằng 0). Dưới đây là cách áp dụng chính quy hóa l_2 cho các trọng số kết nối của một lớp Keras, sử dụng hệ số chính quy hóa 0.01:

```
layer = tf.keras.layers.Dense(100, activation="relu",
                               kernel_initializer="he_normal",
                               kernel_regularizer=tf.keras.regularizers.l2(0.01))
```

Hàm `l2()` trả về một bộ chính quy hóa sẽ được gọi ở mỗi bước trong quá trình huấn luyện để tính toán mất mát chính quy hóa. Sau đó, mất mát này được thêm vào tổng mất mát cuối cùng. Như bạn có thể mong đợi, bạn có thể chỉ cần sử dụng `tf.keras.regularizers.l1()` nếu bạn muốn chính quy hóa l_1 ; nếu bạn muốn cả chính quy hóa l_1 và l_2 , hãy sử dụng `tf.keras.regularizers.l1_l2()` (chỉ định cả hai hệ số chính quy hóa).

Vì bạn thường sẽ muốn áp dụng cùng một bộ chính quy hóa cho tất cả các lớp trong mạng của mình, cũng như sử dụng cùng một hàm kích hoạt và cùng một chiến lược khởi tạo trong tất cả các lớp ẩn, bạn có thể thấy mình lặp lại cùng một đối số. Điều này làm cho mã trở nên xấu và dễ bị lỗi. Để tránh điều này, bạn có thể thử tái cấu trúc mã của mình để sử dụng các vòng lặp. Một lựa chọn khác là sử dụng hàm `functools.partial()` của Python, cho phép bạn tạo một trình bao bọc mỏng cho bất kỳ đối tượng có thể gọi nào, với một số giá trị đối số mặc định:

```
from functools import partial

RegularizedDense = partial(tf.keras.layers.Dense,
                           activation="relu",
                           kernel_initializer="he_normal",
                           kernel_regularizer=tf.keras.regularizers.l2(0.01))

model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
```



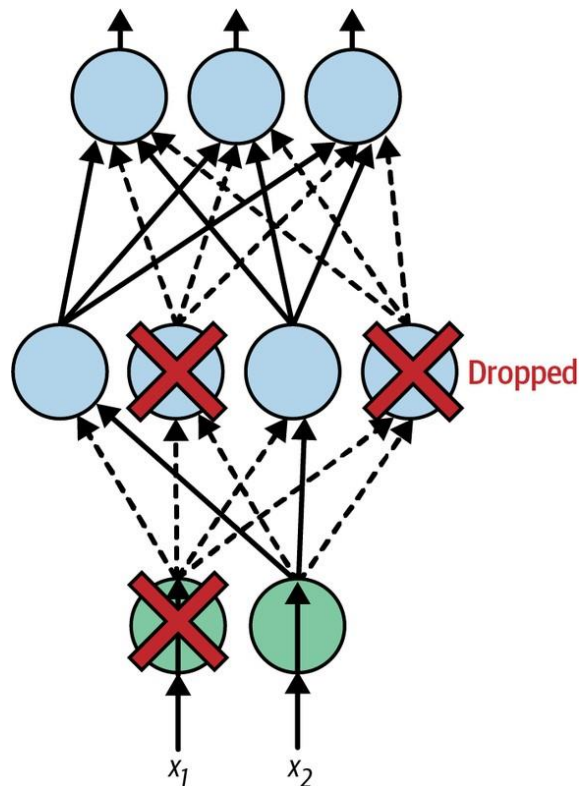
```
RegularizedDense(100),  
RegularizedDense(100),  
RegularizedDense(10, activation="softmax")  
])
```

Dropout

Dropout là một trong những kỹ thuật chính quy hóa phổ biến nhất cho mạng nơ-ron sâu. Nó được đề xuất trong một bài báo của Geoffrey Hinton và cộng sự vào năm 2012 và được mô tả chi tiết hơn trong một bài báo năm 2014 của Nitish Srivastava và cộng sự, và nó đã được chứng minh là rất thành công: nhiều mạng nơ-ron hiện đại sử dụng dropout, vì nó giúp chúng tăng độ chính xác từ 1%–2%. Điều này nghe có vẻ không nhiều, nhưng khi một mô hình đã có độ chính xác 95%, việc tăng độ chính xác 2% có nghĩa là giảm tỷ lệ lỗi gần 40% (từ 5% lỗi xuống khoảng 3%).

Đây là một thuật toán khá đơn giản: ở mỗi bước huấn luyện, mỗi nơ-ron (bao gồm các nơ-ron đầu vào, nhưng luôn trừ các nơ-ron đầu ra) có xác suất p bị “bỏ qua” tạm thời, nghĩa là nó sẽ bị bỏ qua hoàn toàn trong bước huấn luyện này, nhưng nó có thể hoạt động trong bước tiếp theo (xem Hình 11-10). Siêu tham số p được gọi là **tỷ lệ dropout**, và nó thường được đặt giữa 10% và 50%: gần 20%–30% trong mạng nơ-ron hồi quy (xem Chương 15), và gần 40%–50% trong mạng nơ-ron tích chập (xem Chương 14). Sau khi huấn luyện, các nơ-ron không bị bỏ qua nữa. Và đó là tất cả (ngoại trừ một chi tiết kỹ thuật chúng ta sẽ thảo luận ngay sau đây).

Ban đầu thật đáng ngạc nhiên khi kỹ thuật phá hoại này lại hoạt động. Liệu một công ty có hoạt động tốt hơn nếu nhân viên của mình được yêu cầu tung đồng xu mỗi sáng để quyết định có đi làm hay không? Chà, ai biết được; có lẽ là có! Công ty sẽ buộc phải điều chỉnh tổ chức của mình; nó không thể dựa vào bất kỳ một người nào để vận hành máy pha cà phê hoặc thực hiện bất kỳ nhiệm vụ quan trọng nào khác, vì vậy chuyên môn này sẽ phải được phân tán cho nhiều người. Nhân viên sẽ phải học cách hợp tác với nhiều đồng nghiệp của mình, không chỉ một số ít. Công ty sẽ trở nên linh hoạt hơn nhiều. Nếu một người bỏ việc, nó sẽ không tạo ra nhiều khác biệt. Không rõ liệu ý tưởng này có thực sự hoạt động đối với các công ty hay không, nhưng nó chắc chắn hoạt động đối với mạng nơ-ron. Các nơ-ron được huấn luyện với dropout không thể cùng thích nghi với các nơ-ron lân cận của chúng; chúng phải hữu ích nhất có thể một mình. Chúng cũng không thể phụ thuộc quá mức vào chỉ một vài nơ-ron đầu vào; chúng phải chú ý đến từng nơ-ron đầu vào của chúng. Cuối cùng, bạn sẽ có một mạng mạnh mẽ hơn và tổng quát hóa tốt hơn.



Hình 11-10. Với chính quy hóa dropout, ở mỗi lần lặp huấn luyện, một tập hợp con ngẫu nhiên của tất cả các nơ-ron trong một hoặc nhiều lớp — ngoại trừ lớp đầu ra — bị “bỏ qua”; các nơ-ron này xuất ra 0 ở lần lặp này (được biểu diễn bằng các mũi tên đứt nét)

Một cách khác để hiểu sức mạnh của dropout là nhận ra rằng một mạng nơ-ron duy nhất được tạo ra ở mỗi bước huấn luyện. Vì mỗi nơ-ron có thể có mặt hoặc không có mặt, có tổng cộng 2^N mạng có thể có (trong đó N là tổng số nơ-ron có thể bị bỏ qua). Đây là một con số khổng lồ đến mức thực tế không thể có cùng một mạng nơ-ron được lấy mẫu hai lần. Một khi bạn đã chạy 10.000 bước huấn luyện, bạn đã huấn luyện 10.000 mạng nơ-ron khác nhau, mỗi mạng chỉ với một thể hiện huấn luyện. Các mạng nơ-ron này rõ ràng không độc lập vì chúng chia sẻ nhiều trọng số của chúng, nhưng dù sao chúng vẫn khác nhau. Mạng nơ-ron kết quả có thể được xem như một tập hợp trung bình của tất cả các mạng nơ-ron nhỏ hơn này.

Có một chi tiết kỹ thuật nhỏ nhưng quan trọng. Giả sử $p = 75$: trung bình chỉ 25% tổng số nơ-ron hoạt động ở mỗi bước trong quá trình huấn luyện. Điều này có nghĩa là sau khi huấn luyện, một nơ-ron sẽ được kết nối với số nơ-ron đầu vào gấp bốn lần so với trong quá trình huấn luyện. Để bù đắp cho thực tế này, chúng ta cần nhân trọng số kết nối đầu vào của mỗi nơ-ron với bốn trong quá trình huấn luyện. Nếu chúng ta không làm vậy, mạng nơ-ron sẽ không hoạt động tốt vì nó sẽ thấy dữ liệu khác nhau trong và sau quá trình huấn luyện. Tổng quát hơn, chúng ta cần chia trọng số kết nối cho **xác suất giữ lại** ($1 - p$) trong quá trình huấn luyện.

Để triển khai dropout bằng Keras, bạn có thể sử dụng lớp `tf.keras.layers.Dropout`. Trong quá trình huấn luyện, nó ngẫu nhiên bỏ qua một số đầu vào (đặt chúng bằng 0) và chia các đầu vào còn lại cho xác suất giữ lại. Sau khi huấn luyện, nó không làm gì cả; nó chỉ chuyển các đầu vào đến lớp tiếp theo. Đoạn mã sau áp dụng chính quy hóa dropout trước mỗi lớp dày đặc, sử dụng tỷ lệ dropout 0.2:

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(100, activation="relu", kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(100, activation="relu", kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(10, activation="softmax")
])
# ... biên dịch và huấn luyện mô hình
```

Nếu bạn quan sát thấy mô hình bị overfitting, bạn có thể tăng tỷ lệ dropout. Ngược lại, bạn nên thử giảm tỷ lệ dropout nếu mô hình underfits tập huấn luyện. Cũng có thể hữu ích khi tăng tỷ lệ dropout cho các lớp lớn, và giảm nó cho các lớp nhỏ. Hơn nữa, nhiều kiến trúc hiện đại chỉ sử dụng dropout sau lớp ẩn cuối cùng, vì vậy bạn có thể thử điều này nếu dropout toàn diện quá mạnh.

Dropout có xu hướng làm chậm đáng kể sự hội tụ, nhưng nó thường dẫn đến một mô hình tốt hơn khi được điều chỉnh đúng cách. Vì vậy, nó thường rất đáng để dành thêm thời gian và công sức, đặc biệt đối với các mô hình lớn.

Monte Carlo (MC) Dropout

Năm 2016, một bài báo của Yarin Gal và Zoubin Ghahramani đã bổ sung thêm vài lý do chính đáng để sử dụng dropout:

- Thứ nhất, bài báo đã thiết lập một mối liên hệ sâu sắc giữa các mạng dropout (tức là các mạng nơ-ron chứa các lớp Dropout) và suy luận Bayes xấp xỉ, mang lại cho dropout một lý do toán học vững chắc.
- Thứ hai, các tác giả đã giới thiệu một kỹ thuật mạnh mẽ gọi là MC dropout, có thể tăng cường hiệu suất của bất kỳ mô hình dropout đã được huấn luyện nào mà không cần huấn luyện lại hay thậm chí sửa đổi nó một chút nào. Nó cũng cung cấp một thước đo tốt hơn nhiều về sự không chắc chắn của mô hình, và nó có thể được triển khai chỉ trong vài dòng mã.

Nếu tất cả điều này nghe có vẻ giống như một mảnh khố “một mẹo lạ” trên các trang giật tít, thì hãy xem đoạn mã sau. Đó là toàn bộ việc triển khai MC dropout, tăng cường mô hình dropout mà chúng ta đã huấn luyện trước đó mà không cần huấn luyện lại nó:

```
import numpy as np

y_probas = np.stack([model(X_test, training=True)
                     for sample in range(100)])
y_proba = y_probas.mean(axis=0)
```

Lưu ý rằng `model(X)` tương tự như `model.predict(X)` ngoại trừ việc nó trả về một tensor thay vì một mảng NumPy, và nó hỗ trợ đối số `training`. Trong ví dụ mã này, việc đặt `training=True` đảm bảo rằng lớp Dropout vẫn hoạt động, vì vậy tất cả các dự đoán sẽ hơi khác nhau. Chúng ta chỉ thực hiện 100 dự đoán trên tập kiểm tra, và chúng ta tính trung bình của chúng. Cụ thể hơn, mỗi lần gọi mô hình trả về một ma trận với một hàng cho mỗi thể hiện và một cột cho mỗi lớp. Vì có 10.000 thể hiện trong tập kiểm tra và 10 lớp, đây là một ma trận có hình dạng `[10000, 10]`. Chúng ta xếp chồng 100 ma trận như vậy, vì vậy `y_probas` là một mảng 3D có hình dạng `[100, 10000, 10]`. Khi chúng ta tính trung bình trên chiều đầu tiên (`axis=0`), chúng ta nhận được `y_proba`, một mảng có hình dạng `[10000, 10]`, giống như chúng ta sẽ nhận được với một dự đoán duy nhất. Thế thôi!

Tính trung bình trên nhiều dự đoán khi bật dropout mang lại cho chúng ta một ước lượng Monte Carlo thường đáng tin cậy hơn kết quả của một dự đoán duy nhất khi tắt dropout. Ví dụ, hãy xem dự đoán của mô hình cho thể hiện đầu tiên trong tập kiểm tra Fashion MNIST, khi tắt dropout:

```
>>> model.predict(X_test[:1]).round(3)
array([[0.    , 0.    , 0.    , 0.    , 0.    , 0.024, 0.    , 0.132, 0.    ,
        0.844]], dtype=float32)
```

Mô hình khá tự tin (84.4%) rằng hình ảnh này thuộc về lớp 9 (ủng cổ chân). So sánh điều này với dự đoán MC dropout:

```
>>> y_proba[0].round(3)
array([0.    , 0.    , 0.    , 0.    , 0.    , 0.067, 0.    , 0.209, 0.001,
        0.723], dtype=float32)
```

Mô hình vẫn dường như ưu tiên lớp 9, nhưng độ tin cậy của nó đã giảm xuống 72.3%, và xác suất ước tính cho các lớp 5 (sandal) và 7 (giày thể thao) đã tăng lên, điều này có ý nghĩa vì chúng cũng là giày dép.

MC dropout có xu hướng cải thiện độ tin cậy của các ước lượng xác suất của mô hình. Điều này có nghĩa là nó ít có khả năng tự tin nhưng sai, điều này có thể nguy hiểm: chỉ cần tưởng tượng một chiếc xe tự lái tự tin bỏ qua biển báo dừng. Việc biết chính xác những lớp nào khác có nhiều khả năng nhất cũng hữu ích.

Ngoài ra, bạn có thể xem độ lệch chuẩn của các ước lượng xác suất:

```
>>> y_std = y_probas.std(axis=0)

>>> y_std[0].round(3)
array([0.    , 0.    , 0.    , 0.001, 0.    , 0.096, 0.    , 0.162, 0.001,
        0.183], dtype=float32)
```

Rõ ràng có khá nhiều phương sai trong các ước lượng xác suất cho lớp 9: độ lệch chuẩn là 0.183, nên được so sánh với xác suất ước tính 0.723: nếu bạn đang xây dựng một hệ thống nhạy cảm với rủi ro (ví dụ: hệ thống y tế hoặc tài chính), bạn có lẽ sẽ xử lý một dự đoán không chắc chắn như vậy với sự thận trọng cực kỳ. Bạn chắc chắn sẽ không coi nó như một dự đoán tự tin 84.4%.

Độ chính xác của mô hình cũng nhận được một sự tăng nhẹ (rất nhỏ) từ 87.0% lên 87.2%:

```
>>> y_pred = y_proba.argmax(axis=1)

>>> accuracy = (y_pred == y_test).sum() / len(y_test)

>>> accuracy
0.8717
```

Nếu mô hình của bạn chứa các lớp khác hoạt động theo cách đặc biệt trong quá trình huấn luyện (chẳng hạn như các lớp BatchNormalization), thì bạn không nên ép buộc chế độ huấn luyện như chúng ta vừa làm. Thay vào đó, bạn nên thay thế các lớp Dropout bằng lớp MCDropout sau:

```
class MCDropout(tf.keras.layers.Dropout):
    def call(self, inputs, training=False):
        return super().call(inputs, training=True)
```

Ở đây, chúng ta chỉ kế thừa từ lớp Dropout và ghi đè phương thức call() để buộc đối số training của nó thành True (xem Chương 12). Tương tự, bạn có thể định nghĩa một lớp MCAAlphaDropout bằng cách kế thừa từ AlphaDropout thay thế. Nếu bạn đang tạo một mô hình từ đầu, đó chỉ là vấn đề sử dụng MCDropout thay vì Dropout. Nhưng nếu bạn có một mô hình đã được huấn luyện bằng Dropout, bạn cần tạo một mô hình mới giống hệt mô hình hiện có nhưng với MCDropout thay vì Dropout, sau đó sao chép trọng số của mô hình hiện có vào mô hình mới của bạn.

Tóm lại, MC dropout là một kỹ thuật tuyệt vời giúp tăng cường các mô hình dropout và cung cấp các ước lượng không chắc chắn tốt hơn. Và tất nhiên, vì nó chỉ là dropout thông thường trong quá trình huấn luyện, nó cũng hoạt động như một bộ chính quy hóa.

Chính quy hóa Max-Norm

Một kỹ thuật chính quy hóa phổ biến khác cho mạng nơ-ron được gọi là **chính quy hóa max-norm**: đối với mỗi nơ-ron, nó ràng buộc trọng số w của các kết nối đến sao cho $\|w\|_2 \leq r$, trong đó r là siêu tham số max-norm và $\|\cdot\|_2$ là chuẩn l_2 .

Chính quy hóa max-norm không thêm một số hạng mất mát chính quy hóa vào hàm mất mát tổng thể. Thay vào đó, nó thường được triển khai bằng cách tính toán $\|w\|_2$ sau mỗi bước huấn luyện và điều chỉnh tỷ lệ w nếu cần ($w \leftarrow w r / \|w\|_2$). Việc giảm r làm tăng lượng chính quy hóa và giúp giảm overfitting. Chính quy hóa max-norm cũng có thể giúp giảm nhẹ các vấn đề gradient không ổn định (nếu bạn không sử dụng chuẩn hóa theo batch).

Để triển khai chính quy hóa max-norm trong Keras, hãy đặt đối số `kernel_constraint` của mỗi lớp ẩn thành một ràng buộc `max_norm()` với giá trị tối đa thích hợp, như thế này:

```
dense = tf.keras.layers.Dense(  
    100, activation="relu", kernel_initializer="he_normal",  
    kernel_constraint=tf.keras.constraints.max_norm(1.))
```

Sau mỗi lần lặp huấn luyện, phương thức `fit()` của mô hình sẽ gọi đối tượng được trả về bởi `max_norm()`, truyền cho nó trọng số của lớp và nhận lại trọng số đã được điều chỉnh tỷ lệ, sau đó thay thế trọng số của lớp. Như bạn sẽ thấy trong Chương 12, bạn có thể định nghĩa hàm ràng buộc tùy chỉnh của riêng mình nếu cần và sử dụng nó làm `kernel_constraint`. Bạn cũng có thể ràng buộc các số hạng bias bằng cách đặt đối số `bias_constraint`.

Hàm `max_norm()` có đối số `axis` mặc định là 0. Một lớp Dense thường có trọng số có hình dạng [số đầu vào, số nơ-ron], vì vậy việc sử dụng `axis=0` có nghĩa là ràng buộc max-norm sẽ được áp dụng độc lập cho vector trọng số của mỗi nơ-ron. Nếu bạn muốn sử dụng max-norm với các lớp tích chập (xem Chương 14), hãy đảm bảo đặt đối số `axis` của ràng buộc `max_norm()` một cách thích hợp (thường là `axis=[0, 1, 2]`).

Tóm tắt và hướng dẫn thực hành

Trong chương này, chúng ta đã đề cập đến một loạt các kỹ thuật, và bạn có thể tự hỏi mình nên sử dụng kỹ thuật nào. Điều này phụ thuộc vào tác vụ, và chưa có sự đồng thuận rõ ràng, nhưng tôi đã thấy cấu hình trong Bảng 11-3 hoạt động tốt trong hầu hết các trường hợp, mà không yêu cầu nhiều tinh chỉnh siêu tham số. Điều đó nói lên rằng, xin đừng coi những mặc định này là những quy tắc cứng nhắc!

Bảng 11-3. Cấu hình DNN mặc định

Lựa chọn	Giá trị
Khởi tạo trọng số	He initialization
Hàm kích hoạt	ReLU hoặc biến thể (Leaky ReLU, ELU, GELU, Swish, Mish)
Chuẩn hóa	Batch Normalization
Bộ tối ưu hóa	Adam (hoặc Nadam, AdamW)
Lịch trình tốc độ học	Exponential decay hoặc 1cycle
Chính quy hóa	Early stopping, Batch Normalization, Dropout

Nếu mạng là một chồng đơn giản các lớp dày đặc, thì nó có thể tự chuẩn hóa, và bạn nên sử dụng cấu hình trong Bảng 11-4 thay thế.

Bảng 11-4. Cấu hình DNN cho mạng tự chuẩn hóa

Lựa chọn	Giá trị
Khởi tạo trọng số	LeCun normal initialization
Hàm kích hoạt	SELU

Lựa chọn	Giá trị
Chuẩn hóa	(Không cần Batch Normalization)
Bộ tối ưu hóa	Nesterov accelerated gradients
Lịch trình tốc độ học	(Có thể thử 1cycle nếu không dùng Batch Normalization)
Chính quy hóa	Early stopping (tránh l_1 , l_2 , max-norm, Batch Normalization, Dropout thông thường)

Đừng quên chuẩn hóa các đặc trưng đầu vào! Bạn cũng nên cố gắng tái sử dụng các phần của một mạng nơ-ron đã được huấn luyện trước nếu bạn có thể tìm thấy một mạng giải quyết một vấn đề tương tự, hoặc sử dụng tiền huấn luyện không giám sát nếu bạn có nhiều dữ liệu không gắn nhãn, hoặc sử dụng tiền huấn luyện trên một tác vụ phụ trợ nếu bạn có nhiều dữ liệu được gắn nhãn cho một tác vụ tương tự.

Mặc dù các hướng dẫn trước đó sẽ bao gồm hầu hết các trường hợp, nhưng đây là một số ngoại lệ:

- Nếu bạn cần một mô hình thưa thớt, bạn có thể sử dụng chính quy hóa l_1 (và tùy chọn đặt các trọng số rất nhỏ bằng 0 sau khi huấn luyện). Nếu bạn cần một mô hình thưa thớt hơn nữa, bạn có thể sử dụng TensorFlow Model Optimization Toolkit. Điều này sẽ phá vỡ khả năng tự chuẩn hóa, vì vậy bạn nên sử dụng cấu hình mặc định trong trường hợp này.
- Nếu bạn cần một mô hình có độ trễ thấp (một mô hình thực hiện dự đoán nhanh như chớp), bạn có thể cần sử dụng ít lớp hơn, sử dụng hàm kích hoạt nhanh như ReLU hoặc leaky ReLU, và gộp các lớp chuẩn hóa theo batch vào các lớp trước đó sau khi huấn luyện. Có một mô hình thưa thớt cũng sẽ giúp ích. Cuối cùng, bạn có thể muốn giảm độ chính xác dấu phẩy động từ 32 bit xuống 16 hoặc thậm chí 8 bit (xem “Triển khai mô hình sang thiết bị di động hoặc nhúng”). Một lần nữa, hãy kiểm tra TF-MOT.
- Nếu bạn đang xây dựng một ứng dụng nhạy cảm với rủi ro, hoặc độ trễ suy luận không quá quan trọng trong ứng dụng của bạn, bạn có thể sử dụng MC dropout để tăng hiệu suất và nhận được các ước lượng xác suất đáng tin cậy hơn, cùng với các ước lượng không chắc chắn.

Với các hướng dẫn này, bây giờ bạn đã sẵn sàng để huấn luyện các mạng sâu! Tôi hy vọng bạn đã bị thuyết phục rằng bạn có thể đi rất xa chỉ bằng cách sử dụng API Keras tiện lợi. Tuy nhiên, có thể đến lúc bạn cần kiểm soát nhiều hơn nữa; ví dụ, để viết một hàm mất mát tùy chỉnh hoặc để điều chỉnh thuật toán huấn luyện. Đối với những trường hợp như vậy, bạn sẽ cần sử dụng API cấp thấp hơn của TensorFlow, như bạn sẽ thấy trong chương tiếp theo.

Bài tập

1. Vấn đề mà khởi tạo Glorot và khởi tạo He nhằm mục đích khắc phục là gì?
2. Có ổn không khi khởi tạo tất cả các trọng số với cùng một giá trị miễn là giá trị đó được chọn ngẫu nhiên bằng cách sử dụng khởi tạo He?
3. Có ổn không khi khởi tạo các số hạng bias về 0?

4. Trong những trường hợp nào bạn sẽ muốn sử dụng từng hàm kích hoạt chúng ta đã thảo luận trong chương này?
5. Điều gì có thể xảy ra nếu bạn đặt siêu tham số momentum quá gần 1 (ví dụ: 0.99999) khi sử dụng bộ tối ưu hóa SGD?
6. Kể tên ba cách bạn có thể tạo một mô hình thừa thớt.
7. Dropout có làm chậm quá trình huấn luyện không? Nó có làm chậm quá trình suy luận (tức là đưa ra dự đoán trên các thể hiện mới) không? Còn MC dropout thì sao?
8. Thực hành huấn luyện một mạng nơ-ron sâu trên tập dữ liệu hình ảnh CIFAR10: a. Xây dựng một DNN với 20 lớp ẩn, mỗi lớp 100 nơ-ron (quá nhiều, nhưng đó là mục đích của bài tập này). Sử dụng khởi tạo He và hàm kích hoạt Swish. b. Sử dụng tối ưu hóa Nadam và dừng sớm, huấn luyện mạng trên tập dữ liệu CIFAR10. Bạn có thể tải nó bằng `tf.keras.datasets.cifar10.load_data()`. Tập dữ liệu bao gồm 60.000 hình ảnh màu 32×32 pixel (50.000 để huấn luyện, 10.000 để kiểm tra) với 10 lớp, vì vậy bạn sẽ cần một lớp đầu ra softmax với 10 nơ-ron. Hãy nhớ tìm kiếm tốc độ học phù hợp mỗi khi bạn thay đổi kiến trúc hoặc siêu tham số của mô hình. c. Bây giờ hãy thử thêm chuẩn hóa theo batch và so sánh các đường cong học tập: nó có hội tụ nhanh hơn trước không? Nó có tạo ra một mô hình tốt hơn không? Nó ảnh hưởng đến tốc độ huấn luyện như thế nào? d. Thử thay thế chuẩn hóa theo batch bằng SELU, và thực hiện các điều chỉnh cần thiết để đảm bảo mạng tự chuẩn hóa (tức là chuẩn hóa các đặc trưng đầu vào, sử dụng khởi tạo chuẩn LeCun, đảm bảo DNN chỉ chứa một chuỗi các lớp dày đặc, v.v.). e. Thử chính quy hóa mô hình bằng alpha dropout. Sau đó, không huấn luyện lại mô hình của bạn, xem liệu bạn có thể đạt được độ chính xác tốt hơn bằng cách sử dụng MC dropout. f. Huấn luyện lại mô hình của bạn bằng cách sử dụng lập lịch 1 chu kỳ và xem liệu nó có cải thiện tốc độ huấn luyện và độ chính xác của mô hình không.

Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại <https://homl.info/colab3>.